

Week 5 Assignment: Database Design

Yelysei Shkirpan | Capstone Project | September 20, 2026

Built for Cars, Deadly for Walkers: Database Design

This document describes the database behind my capstone project and shows that it works. The eight parts follow the eight items of the assignment. Everything in it can be rebuilt from the repository: github.com/Sor1na/florida-pedestrian-patterns. The SQL is in `sql/`, the test records and tests are in `tests/`, the queries are in `queries/`, and the logs I quote are in `docs/evidence/`.

One thing to know first. The database has not been loaded with real FARS rows yet. That is next week's step in my plan. Every row used here is synthetic, so the numbers in the query results are not findings. They only show that the design does what it should.

1. Project Question, Audience, Scope, and Unit of Observation

Project question

Which combinations of roadway class, posted speed limit, lighting, crossing location, and time of day most frequently characterize fatal pedestrian crashes on Florida's urban roads from 2020 through 2024, and which counties concentrate the most deaths under each pattern?

Intended audience

Safety planners at FDOT's State Safety Office and at Florida's metropolitan planning organizations, for example the Space Coast TPO. They decide which FHWA proven countermeasures (lighting, crossings, medians, speed management) to fund first, and in which counties and corridor types. They need patterns in plain terms that map onto a countermeasure. A second audience is me and anyone who reruns the pipeline: the database also has to make a bad load fail loudly.

Scope

Item	In the database	In the analysis view
Population	Every pedestrian (PER_TYP = 5) with a fatal injury (INJ_SEV = 4) in a Florida crash (STATE = 12). About 3,700 people.	The same people, limited to urban roads (RUR_URB = 2) without interstates and freeways (FUNC_SYS 1-2).
Period	Crash years 2020-2024. The 2024 file is an Annual Report File and may be replaced.	Same.
Geography	All 67 counties. Census block groups where the crash has valid coordinates.	Same.
Sources	NHTSA FARS: accident, vehicle, person, and ptype files (NHTSA, 2026). EPA National Walkability Index (EPA, 2021b). Census county population estimates, Vintage 2025 (U.S. Census Bureau, 2026).	Same.
Left out	Non-fatal crashes, bicyclists, drivers and passengers, parked vehicles, other states. RACE and HISPANIC are never loaded (Shkirpan, 2026, section 10). The day of the month is not loaded either.	Rural crashes and freeway crashes stay in the tables, so county rates count every death, but they are outside the view.

The urban filter lives in a view and not in the tables on purpose. County fatality rates (Objective 3) need every death in the county. The pattern mining needs only the urban, non-freeway subset. One set of tables serves both.

Unit of observation

One pedestrian fatality: a FARS person record coded as a pedestrian (PER_TYP = 5) with a fatal injury (INJ_SEV = 4) in a Florida crash from 2020 through 2024, linked to the crash it happened in, the vehicle that struck it, the county it happened in, and, when the coordinates are valid, the census block group around it.

So the unit is the person, not the crash. A crash that kills two pedestrians gives two observations that share one scene. In the database this unit is one row of the table `pedestrian` and, when the death is in scope, one row of the view `v_ped_transactions`.

What the question demands from the database

I turned the question into six requirements. The rest of the document refers back to them.

#	Requirement	How the design meets it
D1	All five conditions in the question must end up on one row per death.	FARS stores them in three files. Lighting, road class, and hour are on the crash. Posted speed is on the vehicle. Crossing location is on the crash-typing record. Keys and foreign keys make the joins exact; <code>v_ped_transactions</code> does them once.
D2	Count deaths, not crashes.	<code>pedestrian</code> is its own table under <code>crash</code> (one-to-many). Two deaths in one crash are two rows.
D3	Every death lands in a county, and every county has a denominator.	<code>crash.county</code> is mandatory and must exist in county. <code>county_population</code> holds one estimate per county and year. <code>v_county_rate</code> starts from all 67 counties.
D4	"Unknown" must stay visible. A planner should see that the speed limit is unknown in some share of deaths.	FARS unknown codes (8, 9, 98, 99, 998, 999) are legal values. Coded columns are NOT NULL, so NULL never doubles as "unknown".
D5	Years must not mix, and one year must be reloadable.	<code>year</code> is part of every FARS key, because FARS reuses case numbers every year. Deleting a year's crashes removes their children (ON DELETE CASCADE).
D6	The rows describe real deaths. Person-level data stays local.	Three roles. The dashboard role can read aggregates only. Sensitive columns are marked in the data dictionary.

2. Conceptual ER Diagram

Figure 1 is the conceptual model in crow's foot notation. The symbol next to an entity says how many of that entity take part for one instance of the other: `|` exactly one, `|o` zero or one, `}|` one or more, `}o` zero or more. A solid line is an identifying relationship (the child's key contains the parent's key). A dashed line is a non-identifying one (a plain foreign key). The source is `docs/erd_conceptual.mmd`.

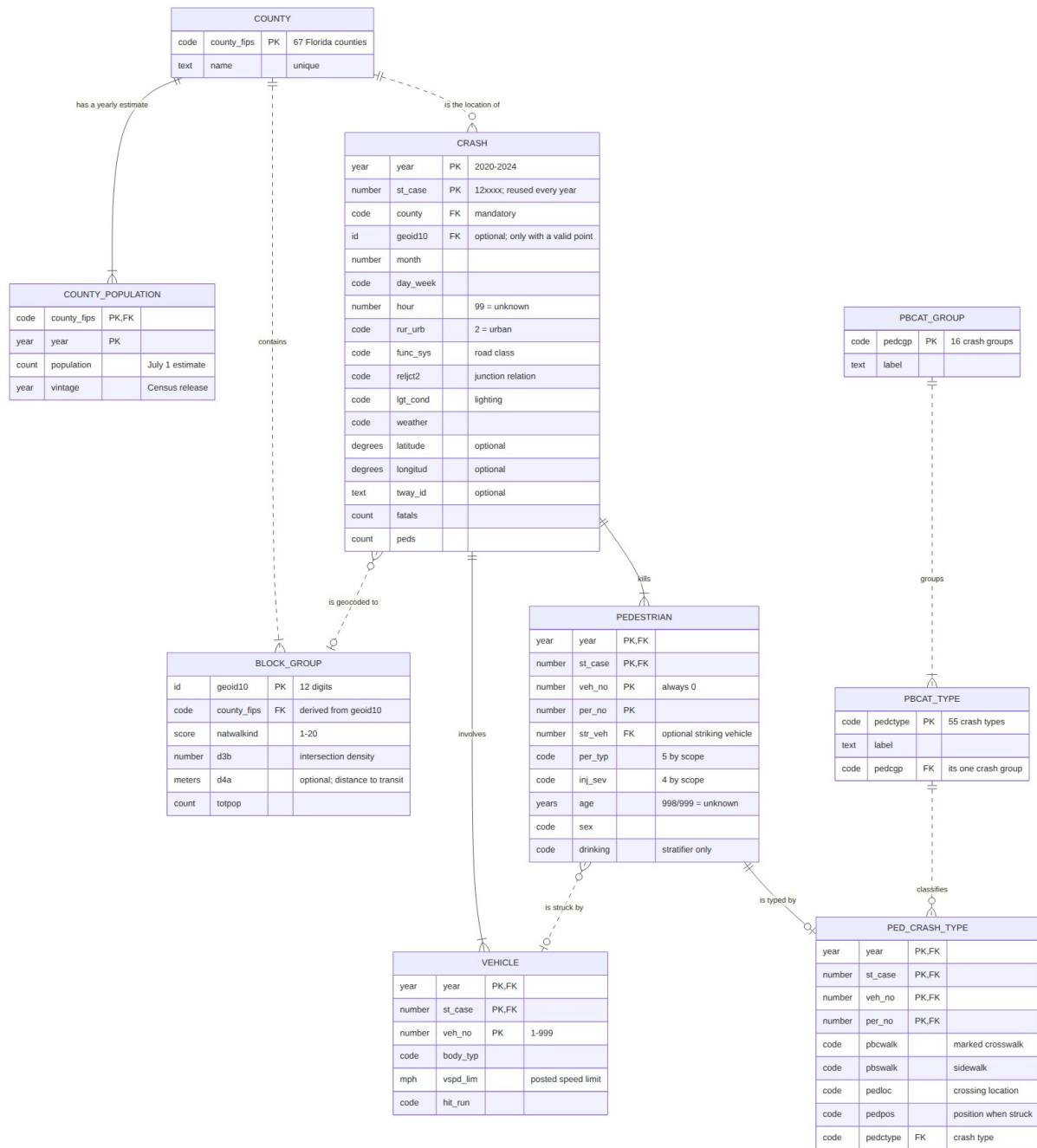


Figure 1. Conceptual ER diagram: nine entities with keys, attributes, cardinality, and optionality.

Entities

Entity	One instance is	Identifier	Attributes and why they are here
CRASH	one fatal crash in Florida, 2020-2024, that killed at least one pedestrian	year + st_case	county, month, day_week, hour (when and where); rur_urb, func_sys, reljct2, lgt_cond, weather (the scene); latitude, longitud, geoid10 (the point); tway_id (corridor label); fatalities, peds (reconciliation)

Entity	One instance is	Identifier	Attributes and why they are here
VEHICLE	one motor vehicle in transport in that crash	year + st_case + veh_no	vspd_lim (FARS records the posted speed per vehicle), body_typ, hit_run
PEDESTRIAN	one person killed while walking: the unit of observation	year + st_case + veh_no (always 0) + per_no	str_veh (link to the striking vehicle); age, sex; drinking (stratifier only); per_typ and inj_sev as scope guards
PED_CRASH_TYPE	the crash-typing (PBCAT) record of one pedestrian	same four attributes as PEDESTRIAN	pbcwalk, pbswalk, pedloc, pedpos (crossing location); pedctype (what happened)
PBCAT_TYPE	one of the 55 pedestrian crash types	pedctype	label; pedcgp (the one group the type belongs to)
PBCAT_GROUP	one of the 16 pedestrian crash groups	pedcgp	label
COUNTY	one of Florida's 67 counties	county_fips	name
COUNTY_POPULATION	one July 1 population estimate for one county and year	county_fips + year	population (rate denominator); vintage (which Census release)
BLOCK_GROUP	one census block group with its walkability score	geoid10	county_fips (derived from the GEOID); natwalkind, d3b, d4a, totpop

Relationships

Relationship	Left to right	Right to left	Why this shape
COUNTY - COUNTY_POPULATION	a county has one or more yearly estimates	an estimate belongs to exactly one county	one denominator per county and year; a new year adds rows, not columns
COUNTY - CRASH	a county is the location of zero or more crashes	a crash happened in exactly one county	mandatory for the crash (D3); a county with no deaths still exists, with rate 0
COUNTY - BLOCK_GROUP	a county contains one or more block groups	a block group lies in exactly one county	lets walkability fall back to the county level
CRASH - BLOCK_GROUP	a crash is geocoded to zero or one block group	a block group holds zero or more crash points	optional: coordinates are sometimes missing, and the link is derived, not from FARS
CRASH - PEDESTRIAN	a crash kills one or more pedestrians	a pedestrian died in exactly one crash	only crashes with a pedestrian death are loaded (D2)
CRASH - VEHICLE	a crash involves one or more vehicles	a vehicle belongs to exactly one crash	FARS only covers crashes with a motor vehicle in transport
PEDESTRIAN - VEHICLE	a pedestrian is struck by zero or one vehicle	a vehicle strikes zero or more pedestrians	FARS names one striking vehicle; the link is optional until I have profiled the real rows
PEDESTRIAN - PED_CRASH_TYPE	a pedestrian has zero or one typing record	a typing record belongs to exactly one pedestrian	a missing record should be a warning, not a failed load
PBCAT_TYPE - PED_CRASH_TYPE	a crash type classifies zero or more typing records	a typing record has exactly one crash type	the type list is closed (55 codes)

Relationship	Left to right	Right to left	Why this shape
PBCAT_GROUP - PBCAT_TYPE	a group has one or more types	a type belongs to exactly one group	the PBCAT hierarchy; this is why the group is not stored per pedestrian

What changed since the first-pass model

I submitted a first-pass conceptual model last week and left two questions for peer reviewers. No comments had arrived when I built this version, so I settled both questions myself.

- **PEDESTRIAN stays narrow.** I asked whether to widen it to a general PERSON entity with drivers and passengers. I decided not to. The question is about roadway conditions, not about drivers. Driver rows would triple the person data and add sensitive attributes that nothing in the project uses. If I ever need them, the key already matches the FARS person key, so widening is an added table or a relaxed CHECK, not a redesign.
- **The crash-to-block-group link stays a nullable foreign key on CRASH.** The proposal had a separate crash_blockgroup table. I kept the foreign key for one practical reason: the rule "no block group without a valid point" then involves one row of one table, so a plain CHECK can enforce it. With a separate table it would need a trigger. The details of the spatial join (method, boundary version, date) are the same for every crash in a run, so they belong to the run and go in etl_log, not on each crash.
- **PBCAT_TYPE and PBCAT_GROUP are new, and PED_CRASH_TYPE lost its crash-group column.** Every crash type belongs to exactly one crash group. Storing both for each pedestrian would let them disagree (part 3).
- **BLOCK_GROUP lost its geometry, and its county is now derived.** The point-in-polygon join runs in geopandas, and the dashboard draws county shapes from its own boundary file. Nothing inside the database uses the polygon, so I dropped it and the PostGIS requirement with it. A fresh machine now needs only a standard PostgreSQL install.
- **Two corrections.** FARS replaced the pickup body types 30 and 31 with 34 in 2020, so test record R1 now uses 34. And the walkability class limits in Appendix B of my proposal were wrong. EPA's limits are 5.75, 10.5, and 15.25 (EPA, 2021a); the view uses those.

3. Logical Relational Schema and Normalization Decisions

The nine entities become nine tables with the same names. Six more tables come from the proposal's architecture: code_lookup (labels for the FARS codes), four pipeline tables (etl_run, source_file, etl_log, validation_result), and data_dictionary. That makes 15 tables. In the notation below, **bold** marks the primary key, an arrow marks a foreign key, and a small circle (°) marks a column that may be NULL.

Table	Columns
county	county_fips , name (unique)
county_population	county_fips -> county, year , population, vintage
block_group	geoid10 , county_fips -> county (generated from geoid10), natwalkind, d3b°, d4a°, totpop
crash	year , st_case , county -> county, month, day_week, hour, rur_urb, func_sys, reljct2, lgt_cond, weather, latitude°, longitud°, geoid10° -> block_group, tway_id°, fatalities, peds
vehicle	year , st_case , veh_no , body_tpy, vspd_lim, hit_run; (year, st_case) -> crash
pedestrian	year , st_case , veh_no , per_no , str_veh°, per_tpy, inj_sev, age, sex, drinking; (year, st_case) -> crash; (year, st_case, str_veh) -> vehicle

Table	Columns
ped_crash_type	year, st_case, veh_no, per_no, pbcwalk, pbswalk, pedloc, pedpos, pedctype -> pbcat_type; (year, st_case, veh_no, per_no) -> pedestrian
pbcat_type	pedctype, label (unique), pedcgp -> pbcat_group
pbcat_group	pedcgp, label (unique)
code_lookup	variable, code, year_from, year_to, label, is_unknown
etl_run	run_id, started_at, finished_at°, git_commit°, status
source_file	file_id, run_id -> etl_run, source, data_year°, release, file_name, url, sha256, retrieved_on; (run_id, file_name) unique
etl_log	log_id, run_id -> etl_run, step, rows_in°, rows_out°, status, message°, logged_at
validation_result	result_id, run_id -> etl_run, check_name, passed, detail°, checked_at; (run_id, check_name) unique
data_dictionary	table_name, column_name, definition, source, allowable_values, null_meaning, sensitivity

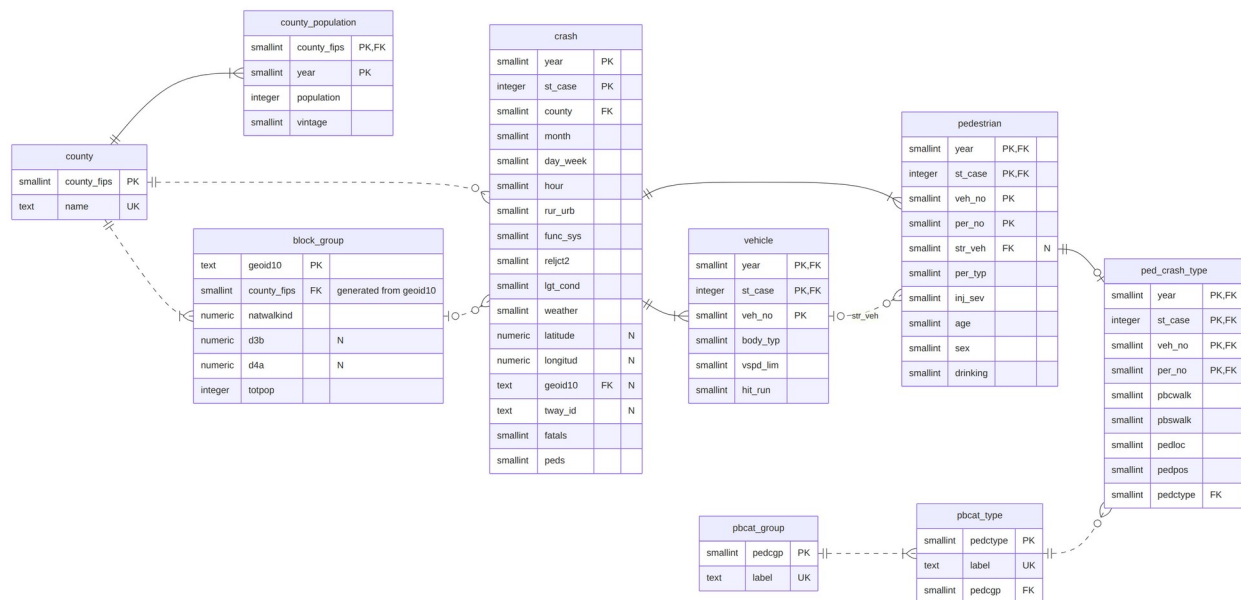


Figure 2. Logical schema, part 1: the nine linked tables with PostgreSQL types (N = nullable, UK = unique). Part 2, the pipeline tables, is docs/erd_logical_pipeline.png.

Normalization decisions

The target is third normal form for everything that is stored, with all derived values in views. These are the decisions, by normal form.

First normal form: one value per cell, no repeating groups. The Census file has five columns, POPESTIMATE2020 to POPESTIMATE2024. My proposal copied that layout (pop2020 ... pop2024 in the county table). That is a repeating group, so county_population now has one row per county and year. A sixth year or a new Census release adds rows and changes no table.

Second normal form: every attribute depends on the whole key. This matters here because most keys are composite. The posted speed limit depends on the vehicle (year, st_case, veh_no), so it sits in vehicle. If it sat in pedestrian, a vehicle that killed two people would store it twice. Lighting and road class depend on the crash alone, so they sit in crash and are not repeated for each pedestrian.

The county name depends on county_fips alone, not on (county_fips, year), so it sits in county and not in county_population.

Third normal form: no attribute depends on another non-key attribute. Three cases came up.

- The crash group depends on the crash type, not on the pedestrian: type 760 always belongs to group 750. FARS delivers both values for every pedestrian, and my first-pass model stored both. Now ped_crash_type stores the type only, and pbcat_type gives the group. The two can no longer disagree.
- The county of a block group depends on its GEOID (digits 3-5). I kept the column, because a foreign key to county is useful. But it is a generated column: PostgreSQL computes it from geoid10, so nobody can type in a county that contradicts the GEOID.
- County names, walkability scores, and code labels are stored once, in their own tables, and reach a crash only through a key.

Where I broke the rules on purpose. Each exception has a reason and a guard.

Exception	Why I accept it	Guard
crash.geoid10 can be derived from latitude and longitud.	The derivation is a spatial join that runs in geopandas, outside the database. Storing the result is the only way to use it in SQL.	CHECK: no block group without coordinates. v_dq_county_block_group_mismatch lists crashes whose block group lies in another county than FARS coded.
crash.fatals and crash.peds overlap with row counts.	They are FARS's own counts. I keep them to reconcile a load, not to analyze.	v_dq_count_mismatch lists crashes with more pedestrian rows than FARS counted.
pedestrian.per_typ, inj_sev, and veh_no are constants (5, 4, 0).	They are scope guards: a CHECK on each makes it impossible to load a bicyclist or a survivor by mistake. veh_no also keeps the key equal to the FARS person key.	CHECK constraints; tests C3, C4, C5.
code_lookup is one table for many code lists, and no foreign key points to it.	A foreign key would need the variable name as an extra column in every data table. Allowed values are enforced by CHECK constraints instead; the table only supplies labels. The two PBCAT lists are different: they have a real hierarchy, so they got real tables and foreign keys.	tests/check_code_domains.py compares every CHECK list with the labels. v_dq_unlabeled_code lists codes in the data without a label.

One more point on keys. In pedestrian, veh_no is always 0, so (year, st_case, per_no) alone would identify a row. Strictly, my four-column primary key is not minimal. I keep it because it is the FARS person key: the raw person and pctype files join on it, and the key would not change if the table were ever widened to all persons.

4. Physical Implementation

The target is PostgreSQL 16, as planned in the proposal. The only extension is btree_gist, which ships with PostgreSQL. One command builds everything: psql -d ped_safety -f sql/00_build_all.sql. The build is a single transaction, so a failure leaves the database as it was.

Script	What it does
sql/00_build_all.sql	Drops and recreates the schema, runs the scripts below in order, prints a summary.

Script	What it does
sql/01_schema.sql	15 tables with all column rules. This is the SQL definition script; it is reproduced in Appendix A.
sql/02_cardinality_triggers.sql	Deferred constraint triggers for the two "one or more" rules. Also in Appendix A.
sql/03_indexes.sql	7 indexes on top of the 21 that the key constraints create.
sql/04_views.sql	3 analysis views and 7 data-quality views.
sql/05_roles.sql	3 group roles and their permissions. No passwords.
sql/06_seed_county.sql, 07_seed_codes.sql	67 counties; 98 code labels; 16 PBCAT groups and 55 PBCAT types.
sql/08_load_data_dictionary.sql	Loads docs/data_dictionary.csv into the database and copies the definitions into column comments.

A fresh build creates 15 tables, 10 views, and 28 indexes. It declares 15 primary keys, 13 foreign keys, 5 unique constraints, 58 CHECK constraints, 1 EXCLUDE constraint, 3 constraint triggers, and 86 NOT NULL columns (log: docs/evidence/01_build.log).

Data types

Type	Used for	Reason
smallint	every FARS code, year, counts, county_fips	codes are small whole numbers; a number cannot carry stray spaces or case differences the way text can
integer	st_case, population, identity keys	st_case has six digits, too large for smallint
numeric(10,8), numeric(11,8)	latitude, longitud	FARS publishes eight decimals; numeric keeps them exactly, float would not
numeric(5,2), numeric(9,3), numeric(9,2)	natwalkind, d3b, d4a	fixed decimals as published by EPA
text	geoid10, names, labels, tway_id, sha256	a GEOID is an identifier with leading zeros, not a number; length and format are set by CHECK, which PostgreSQL recommends over char(n)
boolean	is_unknown, passed	two-valued facts
timestampz, date	pipeline times, retrieved_on	time-zone safe
generated column	block_group.county_fips	computed from geoid10, cannot be typed in

How each business rule is enforced

The three business rules come from my first-pass model. The table shows the constraint behind each rule and the test that proves it (part 6).

Rule	Enforced by	Tests
Rule 1. A crash is identified by year + st_case, and every dependent record carries both.	PRIMARY KEY (year, st_case) on crash; composite foreign keys from vehicle, pedestrian, and ped_crash_type, all including year	A1, A2, B1, B2, B10
Rule 2. A pedestrian has at most one striking vehicle, and it is a vehicle of the same crash.	one nullable column str_veh; FOREIGN KEY (year, st_case, str_veh) to vehicle; CHECK that str_veh is 1-999, so "none" can only be NULL	B3, B4, B5, B6

Rule	Enforced by	Tests
Rule 3. County is mandatory and must exist. Block group is optional and needs valid coordinates.	NOT NULL + FOREIGN KEY on county; nullable FOREIGN KEY on geoid10; CHECKs: both coordinates or neither, inside Florida's bounding box, no block group without a point	B7, B8, B9, C15-C18
Scope: Florida, 2020-2024, pedestrians, fatal.	CHECKs on year, st_case (120001-129999), per_typ = 5, inj_sev = 4, veh_no = 0	C1-C5
Allowed codes for every coded column.	one named CHECK per column, written from the FARS manuals	C6-C14, C19-C21
A crash has one or more pedestrians and one or more vehicles.	deferred constraint triggers (a foreign key cannot say "at least one child")	D1-D4
One label per code per year.	EXCLUDE constraint on code_lookup (variable, code, year range)	C24
Person-level rows stay local.	roles: ped_dashboard may read aggregate views and reference tables only	E1-E6

Referential actions. Children of a crash are deleted with it (ON DELETE CASCADE on vehicle, pedestrian, and ped_crash_type). That is how one year is reloaded when NHTSA replaces the 2024 Annual Report File: delete the year's crashes, load the new file. Every other foreign key uses the default, NO ACTION, so a county, a block group, a crash type, or a striking vehicle cannot be deleted while something points to it.

The "one or more" triggers. A foreign key can say that every pedestrian has a crash. It cannot say that every crash has a pedestrian. For that I wrote one trigger function and attached it as a deferred constraint trigger (PostgreSQL Global Development Group, 2023): it runs at COMMIT. A load can insert the crash first and its children after, as long as they arrive in the same transaction.

Indexes

The 21 indexes behind the primary key, unique, and EXCLUDE constraints already cover every join from a child to its parent. I added seven: one for each foreign key that is not the leading part of a primary key, plus one for reading the log of a run. Two of them are partial indexes that skip NULLs. To be honest about it: with about 3,700 pedestrian rows PostgreSQL will mostly scan the tables anyway. The indexes cost almost nothing at this size, and they start to matter if the scope grows to all states.

Index	On	Serves
crash_county_ix	crash (county)	county rates; which counties concentrate a pattern
crash_geoid10_ix	crash (geoid10) WHERE geoid10 IS NOT NULL	walkability join
pedestrian_striking_vehicle_ix	pedestrian (year, st_case, str_veh) WHERE str_veh IS NOT NULL	the "is struck by" join; the check when a vehicle is deleted
block_group_county_ix	block_group (county_fips)	county-level walkability fallback
ped_crash_type_pedctype_ix	ped_crash_type (pedctype)	crash-type labels
pbcats_type_group_ix	pbcats_type (pedcgp)	crash-group labels
etl_log_run_ix	etl_log (run_id, logged_at)	reading the log of one run in order

Views

The base tables keep FARS codes exactly as published. All binning and labelling happens in views, so changing a bin edge never touches stored data. They are plain views, not materialized, so they are always current.

View	One row is	Purpose
v_ped_transactions	one in-scope pedestrian death	The input of the association mining: each death becomes one transaction, and each binned attribute becomes an item (Tan et al., 2019, chapters 2 and 5). Joins crash, vehicle, pedestrian, typing record, PBCAT labels, and block group, and turns codes into items (road_class, speed_bin, light, crossing_location, time_of_day, and nine more). LEFT JOINs keep a death whose vehicle link, typing record, or block group is missing; the item then reads "unknown".
v_county_rate	one county (all 67)	Deaths, person-years, deaths per 100,000 residents per year, and the unstable-rate flag for counties under 20 deaths.
v_dash_pattern_county	one county and pattern	Aggregated counts for the dashboard. No case numbers, coordinates, or person attributes.
v_dq_* (7 views)	one problem	Checks that need more than one row or table: count mismatches, missing typing records, missing vehicle links, county and block group disagreeing, missing population years, codes without a label, columns missing from the data dictionary. The pipeline copies their row counts into validation_result.

Roles

The roles are NOLOGIN group roles, and the scripts hold no passwords. A person creates a login role by hand, sets the password interactively with \password, and joins a group. ped_etl reads and writes everything. ped_analyst reads everything and writes nothing. ped_dashboard reads the two aggregate views, the reference tables, and the labels. It cannot read pedestrian, crash, or v_ped_transactions. A view runs with its owner's rights, so the dashboard role can read an aggregate without any access to the rows under it.

5. Data Dictionary

The dictionary covers all 135 columns: 100 in the 15 tables and 35 in the three analysis views. It lives in three places that cannot drift apart. `docs/data_dictionary.csv` is the source. The build loads it into the table `data_dictionary` and copies the definitions into column comments. The view `v_dq_undocumented_column` compares that table with the PostgreSQL catalog in both directions; it returned 0 rows (part 6). Types, NULL?, and Key below come from the catalog, not from me typing them. `docs/data_dictionary.md` is the same content, generated by `tools/export_data_dictionary.sql`.

NULL. Coded FARS columns are never NULL: "unknown" is a code. A column that may be NULL says what NULL means. **Sensitivity.** Public: published reference data or counts. Internal: pipeline bookkeeping. Restricted: a case-level value about a real death; it stays in the local database and only aggregates leave it. FARS is public and carries no names, but date, age, sex, and place together could single out a victim, so those columns are marked as quasi-identifiers. **Sources.** Code lists: FARS Analytical User's Manual (NHTSA, 2024, 2025a) and the Pedestrian Bicyclist Crash Typing Manual (NHTSA, 2025b); EPA columns: EPA (2021a, 2021c); population: U.S. Census Bureau (2026).

crash - 17 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
year	smallint	no	PK	Crash year. With <code>st_case</code> it identifies the crash, because FARS starts its case numbers over every year.	FARS ACCIDENT file: YEAR.	2020-2024 (project scope).	Not allowed (NOT NULL).	Restricted (case key)
st_case	integer	no	PK	FARS case number: state code (12) followed by a four-digit sequence. Unique within one year only.	FARS ACCIDENT file: ST_CASE.	120001-129999. The range also enforces the Florida scope (STATE = 12).	Not allowed (NOT NULL).	Restricted (case key)
county	smallint	no	FK > county	County where the crash happened. Foreign key to county. Mandatory, so every death lands in a county rate.	FARS ACCIDENT file: COUNTY.	A <code>county_fips</code> that exists in county. The FARS fillers 0, 997, 998, and 999 have no match and are rejected.	Not allowed (NOT NULL).	Restricted
month	smallint	no		Month of the crash. The day of the month is not loaded: the question does not need it, and leaving it out makes a row harder to match to a news report.	FARS ACCIDENT file: MONTH.	1-12.	Not allowed (NOT NULL).	Restricted (quasi-identifier)
day_week	smallint	no		Day of the week.	FARS ACCIDENT file: DAY_WEEK.	1 = Sunday ... 7 = Saturday. FARS has no unknown code for this element after 2009.	Not allowed (NOT NULL).	Restricted (quasi-identifier)
hour	smallint	no		Hour of the crash on the 24-hour clock. Binned into time of day in <code>v_ped_transactions</code> .	FARS ACCIDENT file: HOUR.	0-23; 99 = unknown.	Not allowed (NOT NULL).	Restricted (quasi-identifier)
rur_urb	smallint	no		Rural or urban land use at the crash site. The analysis view keeps code 2 (urban).	FARS ACCIDENT file: RUR_URB.	1 rural; 2 urban; 6 trafficway not in state inventory; 8 not reported; 9 unknown.	Not allowed (NOT NULL).	Restricted
func_sys	smallint	no		Functional class of the road: the road class in the project question. The analysis view drops 1-2 (interstates and other freeways).	FARS ACCIDENT file: FUNC_SYS.	1 interstate; 2 other freeway or expressway; 3 other principal arterial; 4 minor arterial; 5 major collector; 6	Not allowed (NOT NULL).	Restricted

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
						minor collector; 7 local; 96 not in state inventory; 98 not reported; 99 unknown.		
reljct2	smallint	no		Where the crash happened relative to a junction (crash level). The pedestrian's own location is pedloc in ped_crash_type.	FARS ACCIDENT file: RELJCT2.	1 non-junction; 2 intersection; 3 intersection-related; 4 driveway access; 5 ramp-related; 6 railway crossing; 7 crossover-related; 8 driveway-access-related; 16 shared-use path crossing; 17 acceleration/deceleration lane; 18 through roadway; 19 other location in interchange; 20 entrance/exit ramp; 98 not reported; 99 unknown.	Not allowed (NOT NULL).	Restricted
lgt_cond	smallint	no		Light condition: the lighting in the project question.	FARS ACCIDENT file: LGT_COND.	1 daylight; 2 dark, not lighted; 3 dark, lighted; 4 dawn; 5 dusk; 6 dark, unknown lighting; 7 other; 8 not reported; 9 unknown.	Not allowed (NOT NULL).	Restricted
weather	smallint	no		Prevailing atmospheric condition.	FARS ACCIDENT file: WEATHER.	1 clear; 2 rain; 3 sleet or hail; 4 snow; 5 fog, smog, smoke; 6 severe crosswinds; 7 blowing sand or dirt; 8 other; 10 cloudy; 11 blowing snow; 12 freezing rain; 98 not reported; 99 unknown.	Not allowed (NOT NULL).	Restricted
latitude	numeric(10,8)	yes		Latitude of the crash in decimal degrees. Used once, for the block-group join, and never shown on a map as a point.	FARS ACCIDENT file: LATITUDE.	24.3-31.1 (Florida's bounding box). Must be filled together with longitud.	FARS gave a filler instead of a position: 77.7777 not reported, 88.8888 not available, 99.9999 unknown. All three mean no usable point.	Restricted (quasi-identifier)
longitud	numeric(11,8)	yes		Longitude of the crash in decimal degrees (FARS spelling).	FARS ACCIDENT file: LONGITUD.	-87.7 to -79.8 (Florida's bounding box). Must be filled together with latitude.	FARS gave a filler instead of a position: 777.7777, 888.8888, or 999.9999.	Restricted (quasi-identifier)
geoid10	text	yes	FK > block_group	Census block group the crash point falls in. Foreign key to block_group. Not a FARS field: the pipeline derives it with a point-in-polygon join in geopandas.	Derived by the pipeline from latitude, longitud, and the EPA block-group polygons.	A geoid10 that exists in block_group. Allowed only when the coordinates are filled (CHECK).	Not geocoded: the crash has no usable point, or the point falls outside every Florida block group.	Restricted (quasi-identifier)

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
							The death still counts in its county.	
tway_id	text	yes		Name or route number of the road, as written by the police. Used for corridor labels after aggregation.	FARS ACCIDENT file: TWAY_ID.	Free text, trimmed, not empty.	FARS left the trafficway identifier blank.	Restricted (quasi-identifier)
fatals	smallint	no		Number of people who died in the crash, all person types. Reconciliation only: it must be at least the number of pedestrian rows loaded for the crash.	FARS ACCIDENT file: FATALS.	1-99.	Not allowed (NOT NULL).	Restricted
peds	smallint	no		Number of people in the crash who were not in a motor vehicle (pedestrians, cyclists, and others, including survivors). Reconciliation only.	FARS ACCIDENT file: PEDS.	1-99.	Not allowed (NOT NULL).	Restricted

vehicle - 6 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
year	smallint	no	FK > crash, PK	Crash year. Part of the primary key and of the foreign key to crash.	FARS VEHICLE file: YEAR.	2020-2024, through the foreign key.	Not allowed (NOT NULL).	Restricted (case key)
st_case	integer	no	FK > crash, PK	Case number of the crash the vehicle was in. Part of the primary key and of the foreign key to crash.	FARS VEHICLE file: ST_CASE.	A (year, st_case) that exists in crash.	Not allowed (NOT NULL).	Restricted (case key)
veh_no	smallint	no	PK	Number of the vehicle inside its crash. Part of the primary key. Only motor vehicles in transport are loaded.	FARS VEHICLE file: VEH_NO.	1-999. 0 is reserved for people outside vehicles.	Not allowed (NOT NULL).	Restricted (case key)
body_typ	smallint	no		Body type of the vehicle. Grouped into vehicle classes in v_ped_transactions.	FARS VEHICLE file: BODY_TYP.	1-99 (NHTSA body type codes, not contiguous); 98 not reported; 99 unknown. Since 2020 pickups are 34; codes 30 and 31 are no longer used.	Not allowed (NOT NULL).	Restricted
vspd_lim	smallint	no		Posted speed limit, in mph, on the road this vehicle was traveling: the posted speed in the project question. FARS records it per vehicle, which is why it lives here and not in crash.	FARS VEHICLE file: VSPD_LIM.	0 = no statutory limit; 5-95 mph; 98 not reported; 99 unknown.	Not allowed (NOT NULL).	Restricted
hit_run	smallint	no		Whether the vehicle left the scene.	FARS VEHICLE file: HIT_RUN.	0 no; 1 yes; 9 unknown.	Not allowed (NOT NULL).	Restricted

pedestrian - 10 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
year	smallint	no	FK > crash, FK > vehicle, PK	Crash year. Part of the primary key and of both foreign keys.	FARS PERSON file: YEAR.	2020-2024, through the foreign key.	Not allowed (NOT NULL).	Restricted (case key)
st_case	integer	no	FK > crash, FK > vehicle, PK	Case number of the crash the person died in. Part of the primary key and of both foreign keys.	FARS PERSON file: ST_CASE.	A (year, st_case) that exists in crash.	Not allowed (NOT NULL).	Restricted (case key)
veh_no	smallint	no	PK	Vehicle number of the person. Always 0, which is how FARS marks a person outside any vehicle. Kept in the key so the key matches the FARS person key.	FARS PERSON file: VEH_NO.	0 only.	Not allowed (NOT NULL).	Restricted (case key)
per_no	smallint	no	PK	Number of the person among the people outside vehicles in that crash. Part of the primary key.	FARS PERSON file: PER_NO.	1-999.	Not allowed (NOT NULL).	Restricted (case key)
str_veh	smallint	yes	FK > vehicle	Number of the vehicle that struck the pedestrian. With year and st_case it is a foreign key to vehicle, so the striking vehicle is always a vehicle of the same crash.	FARS PERSON file: STR_VEH.	1-999, and the vehicle must exist in the same crash. 0 is rejected.	No striking vehicle in this database: FARS coded 0, or the number names a vehicle that is not in the VEHICLE file. The death is kept; its speed-limit, vehicle-class, and hit-and-run items read unknown.	Restricted
per_typ	smallint	no		Person type. Kept as a guard on the scope: the CHECK accepts pedestrians only.	FARS PERSON file: PER_TYP.	5 (pedestrian) only.	Not allowed (NOT NULL).	Restricted
inj_sev	smallint	no		Injury severity. Kept as a guard on the scope: the CHECK accepts fatal injuries only.	FARS PERSON file: INJ_SEV.	4 (fatal injury) only.	Not allowed (NOT NULL).	Restricted
age	smallint	no		Age in years. Used in four groups only (under 18, 18-34, 35-64, 65 or older).	FARS PERSON file: AGE.	0-120; 998 not reported; 999 unknown.	Not allowed (NOT NULL).	Restricted (quasi-identifier)
sex	smallint	no		Sex as recorded by the police.	FARS PERSON file: SEX.	1 male; 2 female; 8 not reported; 9 unknown. (NHTSA withdrew a code 3 from the 2022 file in 2025.)	Not allowed (NOT NULL).	Restricted (quasi-identifier)
drinking	smallint	no		Whether the police reported alcohol involvement for this person. Sensitive: used only to check that the roadway patterns also	FARS PERSON file: DRINKING.	0 no; 1 yes; 8 not reported; 9 unknown.	Not allowed (NOT NULL).	Restricted (sensitive attribute)

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
				hold without these cases, never as a rule condition and never in a published table.				

ped_crash_type - 9 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
year	smallint	no	FK > pedestrian, PK	Crash year. Part of the primary key and of the foreign key to pedestrian.	FARS PBTYP file: YEAR.	2020-2024, through the foreign key.	Not allowed (NOT NULL).	Restricted (case key)
st_case	integer	no	FK > pedestrian, PK	Case number. Part of the primary key and of the foreign key to pedestrian.	FARS PBTYP file: ST_CASE.	A pedestrian key that exists in pedestrian.	Not allowed (NOT NULL).	Restricted (case key)
veh_no	smallint	no	FK > pedestrian, PK	Always 0 (person outside a vehicle). Part of the primary key and of the foreign key to pedestrian.	FARS PBTYP file: VEH_NO.	0, through the foreign key.	Not allowed (NOT NULL).	Restricted (case key)
per_no	smallint	no	FK > pedestrian, PK	Person number. Part of the primary key and of the foreign key to pedestrian. Because the primary key equals the pedestrian key, one pedestrian has at most one typing record.	FARS PBTYP file: PER_NO.	1-999, through the foreign key.	Not allowed (NOT NULL).	Restricted (case key)
pbcwalk	smallint	no		Whether a marked crosswalk was present at the crash site.	FARS PBTYP file: PBCWALK.	0 none noted; 1 yes; 9 unknown.	Not allowed (NOT NULL).	Restricted
pbswalk	smallint	no		Whether a sidewalk was present at the crash site.	FARS PBTYP file: PBSWALK.	0 none noted; 1 yes; 9 unknown.	Not allowed (NOT NULL).	Restricted
pedloc	smallint	no		Where the pedestrian was relative to an intersection: the crossing location in the project question.	FARS PBTYP file: PEDLOC.	1 at intersection; 2 intersection-related; 3 not at intersection; 4 non-trafficway location; 9 unknown. Code 7 (not a pedestrian) is rejected.	Not allowed (NOT NULL).	Restricted
pedpos	smallint	no		Where the pedestrian was on or near the road when struck.	FARS PBTYP file: PEDPOS.	1 intersection area; 2 crosswalk area; 3 travel lane; 4 paved shoulder, bike lane, or parking lane; 5 sidewalk, shared-use path, or driveway access; 6 unpaved right-of-way; 7 non-trafficway driveway; 8 non-trafficway parking lot or other; 9 other or unknown. Code 77 (not a pedestrian) is rejected.	Not allowed (NOT NULL).	Restricted
pedctype	smallint	no	FK > pbcat_type	Pedestrian crash type: what the pedestrian and the driver were doing. Foreign key to pbcat_type, which also gives the crash group.	FARS PBTYP file: PEDCTYPE.	A pedctype that exists in pbcat_type (55 types). Code 0 (not a pedestrian) is rejected.	Not allowed (NOT NULL).	Restricted

county - 2 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
county_fips	smallint	no	PK	County code. Primary key. FARS writes the same number in its COUNTY element.	U.S. Census Bureau FIPS county codes (equal to the GSA codes FARS uses, in Florida).	The 67 Florida codes: odd numbers 1-133 except 25, plus 86 (Miami-Dade). CHECK 1-133; the seed script fixes the exact set.	Not allowed (NOT NULL).	Public
name	text	no	UQ	County name without the word County.	U.S. Census Bureau.	67 distinct names; unique; no leading or trailing spaces.	Not allowed (NOT NULL).	Public

county_population - 4 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
county_fips	smallint	no	FK > county, PK	County the estimate belongs to. Part of the primary key; foreign key to county.	Census county population totals, Vintage 2025: COUNTY.	A county_fips that exists in county.	Not allowed (NOT NULL).	Public
year	smallint	no	PK	Year of the July 1 estimate. Part of the primary key.	Census Vintage 2025: the year in POPESTIMATE2020 ... POPESTIMATE2024 (columns unpivoted to rows).	2020-2024.	Not allowed (NOT NULL).	Public
population	integer	no		Resident population on July 1 of that year. Denominator of every fatality rate.	Census Vintage 2025: POPESTIMATE<year>.	Whole number greater than 0.	Not allowed (NOT NULL).	Public
vintage	smallint	no		Census release the number came from (2025 = Vintage 2025). Census revises the whole series every year, so the release has to be on record.	Set by the pipeline from the file name (CO-EST2025-ALLDATA).	From the estimate year up to 2030; an estimate for year Y first appears in Vintage Y.	Not allowed (NOT NULL).	Public

block_group - 6 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
geoid10	text	no	PK	Census block group identifier in 2010 geography: state (2 digits) + county (3) + tract (6) + block group (1). Primary key.	EPA Smart Location Database v3: GEOID10.	12 digits starting with 12 (Florida); stored as text so leading zeros survive.	Not allowed (NOT NULL).	Public
county_fips	smallint	no	FK > county	County the block group lies in. Generated by the database from digits 3-5 of geoid10, so it	Derived from geoid10.	A county_fips that exists in county.	Not allowed (NOT NULL; always	Public

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
				cannot be typed in and cannot disagree with the GEOID. Foreign key to county.			computed).	
natwalkind	numeric(5,2)	no		National Walkability Index: how much the built environment favors walking, from 1 (least) to 20 (most). Binned into EPA's four classes in v_ped_transactions.	EPA Smart Location Database v3: NatWalkInd.	1.00-20.00.	Not allowed (NOT NULL).	Public
d3b	numeric(9,3)	yes		Street intersection density: weighted intersections per square mile, with auto-oriented intersections left out.	EPA Smart Location Database v3: D3B.	0 or more.	EPA published no value for this block group.	Public
d4a	numeric(9,2)	yes		Distance from the population-weighted center of the block group to the nearest transit stop, in meters.	EPA Smart Location Database v3: D4A.	0 or more. EPA's filler -99999 is rejected by a CHECK.	No transit stop within three-quarters of a mile (EPA writes -99999). It is not a distance, so it is stored as NULL and never averaged.	Public
totpop	integer	no		Population of the block group as shipped with the index (2018 American Community Survey).	EPA Smart Location Database v3: TotPop.	Whole number, 0 or more.	Not allowed (NOT NULL).	Public

pbcat_group - 2 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
pedcgp	smallint	no	PK	Pedestrian crash group: the broad scenario (for example 750 = crossing roadway, vehicle not turning). Primary key.	FARS/CRSS Pedestrian Bicyclist Crash Typing Manual: PEDCGP.	The 16 pedestrian groups 100-990. Group 0 (not a pedestrian) is left out.	Not allowed (NOT NULL).	Public
label	text	no	UQ	Name of the crash group.	Pedestrian Bicyclist Crash Typing Manual.	Text; unique.	Not allowed (NOT NULL).	Public

pbcat_type - 3 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
pedctype	smallint	no	PK	Pedestrian crash type: the detailed scenario (for example 760 = pedestrian failed to yield). Primary key.	Pedestrian Bicyclist Crash Typing Manual: PEDCTYPE.	The 55 pedestrian types 120-910. Type 0 (not a pedestrian) is left out.	Not allowed (NOT NULL).	Public
label	text	no	UQ	Name of the crash type.	Pedestrian Bicyclist Crash Typing Manual.	Text; unique.	Not allowed (NOT NULL).	Public

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
pedcgp	smallint	no	FK > pbcats_group	The one crash group this type belongs to. Foreign key to pbcats_group. This is why ped_crash_type stores the type only.	Pedestrian Bicyclist Crash Typing Manual (type-to-group table).	A pedcgp that exists in pbcats_group.	Not allowed (NOT NULL).	Public

code_lookup - 6 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
variable	text	no	PK	FARS element the code belongs to, in lower case (for example lgt_cond). Part of the primary key.	FARS Analytical User's Manual.	Lower-case text, not empty.	Not allowed (NOT NULL).	Public
code	smallint	no	PK	The numeric code as it appears in the FARS files. Part of the primary key.	FARS Analytical User's Manual.	Any code listed for that element.	Not allowed (NOT NULL).	Public
year_from	smallint	no	PK	First crash year in which the label is valid. Part of the primary key.	FARS Analytical User's Manual (code lists are given by year range).	2020-2024. Default 2020.	Not allowed (NOT NULL).	Public
year_to	smallint	no		Last crash year in which the label is valid.	FARS Analytical User's Manual.	year_from-2024. Default 2024. An EXCLUDE constraint stops two labels for one code from overlapping in time.	Not allowed (NOT NULL).	Public
label	text	no		What the code means, in NHTSA's wording.	FARS Analytical User's Manual; Pedestrian Bicyclist Crash Typing Manual.	Text, not empty.	Not allowed (NOT NULL).	Public
is_unknown	boolean	no		True when the code means not reported or unknown. Lets one query measure missingness for any element.	Set by hand from the manual when the label is entered.	true / false. Default false.	Not allowed (NOT NULL).	Public

etl_run - 5 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
run_id	integer	no	PK	Number of the pipeline run. Primary key, generated by the database.	Generated (identity column).	Positive whole number.	Not allowed (NOT NULL).	Internal
started_at	timestampz	no		When the run started.	Pipeline clock.	Timestamp with time zone. Default: now.	Not allowed (NOT NULL).	Internal
finished_at	timestampz	yes		When the run ended.	Pipeline clock.	Not before started_at. Filled exactly when status is not running (CHECK).	The run is still going.	Internal
git_commit	text	yes		Git commit of the code that ran, so a load can	git rev-parse HEAD, read by	7-40 lower-case hexadecimal	The code was run	Internal

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
				be traced to the exact scripts.	the pipeline.	characters.	outside a Git checkout.	
status	text	no		Outcome of the run.	Set by the pipeline.	running; succeeded; failed. Default running.	Not allowed (NOT NULL).	Internal

source_file - 9 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
file_id	integer	no	PK	Number of the manifest entry. Primary key, generated by the database.	Generated (identity column).	Positive whole number.	Not allowed (NOT NULL).	Internal
run_id	integer	no	FK > etl_run, UQ	Run that downloaded or read the file. Foreign key to etl_run.	Pipeline.	A run_id that exists in etl_run.	Not allowed (NOT NULL).	Internal
source	text	no		Which of the three sources the file belongs to.	Pipeline.	FARS; EPA_SLD; CENSUS_POPEST.	Not allowed (NOT NULL).	Internal
data_year	smallint	yes		Crash year the file covers.	Pipeline, from the file name.	2020-2024. Must be filled for FARS files and empty for the others (CHECK).	The file is not yearly (EPA and Census files).	Internal
release	text	no		Release of the file, for example Final, Annual Report File, v3.0, or Vintage 2025. The 2024 FARS file is an Annual Report File and will be replaced.	Publisher's download page.	Text.	Not allowed (NOT NULL).	Internal
file_name	text	no	UQ	Name of the downloaded file.	Pipeline.	Text; unique within one run.	Not allowed (NOT NULL).	Internal
url	text	no		Address the file was downloaded from.	Pipeline.	Must start with https://.	Not allowed (NOT NULL).	Internal
sha256	text	no		SHA-256 checksum of the file as downloaded. Pins the exact version that was loaded.	Computed by the pipeline.	64 lower-case hexadecimal characters.	Not allowed (NOT NULL).	Internal
retrieved_on	date	no		Date of the download.	Pipeline clock.	Date.	Not allowed (NOT NULL).	Internal

etl_log - 8 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
log_id	integer	no	PK	Number of the log line. Primary key, generated by the database.	Generated (identity column).	Positive whole number.	Not allowed (NOT NULL).	Internal

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
run_id	integer	no	FK > etl_run	Run the line belongs to. Foreign key to etl_run.	Pipeline.	A run_id that exists in etl_run.	Not allowed (NOT NULL).	Internal
step	text	no		Name of the pipeline step, for example filter person file to Florida pedestrian fatalities.	Pipeline.	Text.	Not allowed (NOT NULL).	Internal
rows_in	integer	yes		Rows that entered the step.	Pipeline.	0 or more.	The step does not process rows (a download, for example).	Internal
rows_out	integer	yes		Rows that left the step.	Pipeline.	0 or more.	The step does not process rows.	Internal
status	text	no		How the step ended.	Pipeline.	ok; warning; error.	Not allowed (NOT NULL).	Internal
message	text	yes		Detail for a warning or an error.	Pipeline.	Free text.	Nothing to add.	Internal
logged_at	timestamptz	no		When the line was written.	Database clock.	Timestamp with time zone. Default: now.	Not allowed (NOT NULL).	Internal

validation_result - 6 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
result_id	integer	no	PK	Number of the result. Primary key, generated by the database.	Generated (identity column).	Positive whole number.	Not allowed (NOT NULL).	Internal
run_id	integer	no	FK > etl_run, UQ	Run the check belongs to. Foreign key to etl_run.	Pipeline.	A run_id that exists in etl_run.	Not allowed (NOT NULL).	Internal
check_name	text	no	UQ	Name of the check, for example yearly count within 1% of NHTSA total. Unique within one run.	Pipeline (pandera checks and the v_dq_views).	Text.	Not allowed (NOT NULL).	Internal
passed	boolean	no		Whether the check passed. A failed check stops the load.	Pipeline.	true / false.	Not allowed (NOT NULL).	Internal
detail	text	yes		Numbers behind the verdict, for example 15 of 15 links resolve.	Pipeline.	Free text.	Nothing to add.	Internal
checked_at	timestamptz	no		When the check ran.	Database clock.	Timestamp with time zone. Default: now.	Not allowed (NOT NULL).	Internal

data_dictionary - 7 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
table_name	text	no	PK	Table or view the entry describes. Part of the primary key.	docs/data_dictionary.csv.	A table or view in this database (v_dq_undocumented_column reports any mismatch).	Not allowed (NOT NULL).	Public
column_name	text	no	PK	Column the entry describes. Part of the primary key.	docs/data_dictionary.csv.	A column of that table or view.	Not allowed (NOT NULL).	Public
definition	text	no		What the column means and why it is here.	docs/data_dictionary.csv.	Text, not empty.	Not allowed (NOT NULL).	Public
source	text	no		Where the value comes from: source file and element, or how it is derived.	docs/data_dictionary.csv.	Text, not empty.	Not allowed (NOT NULL).	Public
allowable_values	text	no		The values the column accepts, in words; the constraints enforce them.	docs/data_dictionary.csv.	Text, not empty.	Not allowed (NOT NULL).	Public
null_meaning	text	no		What an empty value means, or that the column cannot be empty.	docs/data_dictionary.csv.	Text, not empty.	Not allowed (NOT NULL).	Public
sensitivity	text	no		How the column may be shared.	docs/data_dictionary.csv.	Starts with Public, Internal, or Restricted (CHECK).	Not allowed (NOT NULL).	Public

v_ped_transactions (view) - 19 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
year	smallint	view		Crash year of the death.	pedestrian.year.	2020-2024.	Never empty.	Restricted
st_case	integer	view		Case number of the death.	pedestrian.st_case.	120001-129999.	Never empty.	Restricted
per_no	smallint	view		Person number of the death.	pedestrian.per_no.	1-999.	Never empty.	Restricted
county_fips	smallint	view		County of the crash.	crash.county.	One of the 67 county codes.	Never empty.	Restricted
county_name	text	view		Name of that county.	county.name.	One of the 67 county names.	Never empty.	Restricted
road_class	text	view		Road class item.	crash.func_sys: 3 -> principal arterial; 4 -> minor arterial; 5, 6 -> collector; 7 -> local road; 96, 98, 99 -> unknown. Codes 1-2 are filtered out.	principal arterial; minor arterial; collector; local road; unknown.	Never empty: a missing source value becomes the item unknown.	Restricted
speed_bin	text	view		Posted speed limit item, from the striking vehicle.	vehicle.vspd_lim through pedestrian.str_veh.	30 mph or less; 35-40 mph; 45-50 mph; 55 mph or more; no statutory limit; unknown (98, 99, or no striking-vehicle link).	Never empty: a missing source value becomes the item unknown.	Restricted

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
light	text	view		Lighting item.	crash.lgt_cond: 1 -> daylight; 2 -> dark, not lighted; 3 -> dark, lighted; 4, 5 -> dawn or dusk; 6 -> dark, lighting unknown; 7, 8, 9 -> unknown.	daylight; dark, not lighted; dark, lighted; dawn or dusk; dark, lighting unknown; unknown.	Never empty: a missing source value becomes the item unknown.	Restricted
crossing_location	text	view		Crossing location item.	ped_crash_type.pedloc.	at intersection; intersection-related; not at intersection; non-trafficway; unknown (code 9 or no typing record).	Never empty: a missing source value becomes the item unknown.	Restricted
crosswalk	text	view		Marked crosswalk item.	ped_crash_type.pbcwalk.	marked crosswalk; no crosswalk noted; unknown.	Never empty: a missing source value becomes the item unknown.	Restricted
time_of_day	text	view		Time of day item.	crash.hour.	late night (0-5); morning (6-9); midday (10-15); evening (16-19); night (20-23); unknown (99).	Never empty: a missing source value becomes the item unknown.	Restricted
day_type	text	view		Weekday or weekend.	crash.day_week: 1 and 7 -> weekend.	weekday; weekend.	Never empty: a missing source value becomes the item unknown.	Restricted
junction	text	view		Junction item at crash level.	crash.reljct2: 1 -> non-junction; 2 -> intersection; 3 -> intersection-related; 4, 8 -> driveway; 98, 99 -> unknown; the rest -> other junction type.	non-junction; intersection; intersection-related; driveway; other junction type; unknown.	Never empty: a missing source value becomes the item unknown.	Restricted
crash_group	text	view		PBCAT crash group of the death.	pbcac_group.label through ped_crash_type.pedctype and pbcac_type.	One of the 16 group labels; unknown when there is no typing record.	Never empty: a missing source value becomes the item unknown.	Restricted
vehicle_class	text	view		Class of the striking vehicle.	vehicle.body_typ, grouped by code range.	passenger car; SUV; van; pickup; other light truck; bus; medium or heavy truck; motorcycle; other vehicle; unknown.	Never empty: a missing source value becomes the item unknown.	Restricted
hit_and_run	text	view		Whether the striking vehicle left the scene.	vehicle.hit_run.	hit-and-run; driver stayed; unknown.	Never empty: a	Restricted

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
							missing source value becomes the item unknown.	
age_group	text	view		Age group of the pedestrian.	pedestrian.age.	under 18; 18-34; 35-64; 65 or older; unknown (998, 999).	Never empty: a missing source value becomes the item unknown.	Restricted
walkability	text	view		EPA walkability class of the block group around the crash.	block_group.natwalkind through crash.geoid10, cut at EPA's class limits 5.75, 10.5, and 15.25.	least walkable; below average; above average; most walkable; unknown (not geocoded).	Never empty: a missing source value becomes the item unknown.	Restricted
alcohol_stratum	text	view		Alcohol stratum. For the sensitivity check only; never a rule condition.	pedestrian.drinking.	alcohol reported; no alcohol reported; unknown (8, 9).	Never empty: a missing source value becomes the item unknown.	Restricted (sensitive attribute)

v_county_rate (view) - 8 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
county_fips	smallint	view		County code. One row for each of the 67 counties, with or without deaths.	county.county_fips.	One of the 67 county codes.	Never empty.	Public once aggregated
county_name	text	view		County name.	county.name.	One of the 67 county names.	Never empty.	Public once aggregated
deaths_all	bigint	view		Pedestrian deaths in the county, 2020-2024, all roads.	Count of pedestrian rows by crash.county.	0 or more.	Never empty.	Public once aggregated
deaths_in_scope	bigint	view		Deaths inside the analysis scope (urban, not interstate or freeway).	Same count with rur_urb = 2 and func_sys not 1-2.	0 or more; never more than deaths_all.	Never empty.	Public once aggregated
person_years	bigint	view		Sum of the yearly population estimates: the rate denominator.	Sum of county_population.population.	Greater than 0.	No population estimate is loaded for the county (listed by v_dq_population_gap).	Public once aggregated
years_with_population	bigint	view		How many of the five yearly estimates are loaded. Anything under 5 means the rate is overstated.	Count of county_population rows.	0-5.	Never empty.	Public once aggregated
rate_per_100k	numeric	view		Pedestrian deaths per 100,000 residents per	Computed.	0 or more, two decimals.	No population	Public once

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
_per_year				year: 100000 x deaths_all / person_years.			estimate is loaded for the county.	aggregated
unstable_rate	boolean	view		True when the county has fewer than 20 deaths. The dashboard then shows the count and flags the rate (proposal, section 9).	Computed from deaths_all.	true / false.	Never empty.	Public once aggregated

v_dash_pattern_county (view) - 8 columns

Column	Type	NULL?	Key	Definition	Source	Allowable values	Meaning of NULL	Sensitivity
county_fips	smallint	view		County code.	v_ped_transactions.county_fips.	One of the 67 county codes.	Never empty.	Public once aggregated
county_name	text	view		County name.	v_ped_transactions.county_name.	One of the 67 county names.	Never empty.	Public once aggregated
road_class	text	view		Road class item.	v_ped_transactions.road_class.	Same values as in v_ped_transactions.	Never empty.	Public once aggregated
speed_bin	text	view		Posted speed limit item.	v_ped_transactions.speed_bin.	Same values as in v_ped_transactions.	Never empty.	Public once aggregated
light	text	view		Lighting item.	v_ped_transactions.light.	Same values as in v_ped_transactions.	Never empty.	Public once aggregated
crossing_location	text	view		Crossing location item.	v_ped_transactions.crossing_location.	Same values as in v_ped_transactions.	Never empty.	Public once aggregated
time_of_day	text	view		Time of day item.	v_ped_transactions.time_of_day.	Same values as in v_ped_transactions.	Never empty.	Public once aggregated
deaths	bigint	view		Number of in-scope deaths in that county under that combination of the five items.	Count of v_ped_transactions rows.	1 or more.	Never empty.	Public once aggregated

6. Test Records and Evidence That the Constraints Behave

All tests ran on a real server: PostgreSQL 16.13 (Ubuntu 16.13-0ubuntu0.24.04.1). The logs are in docs/evidence/. Anyone can reproduce them with `bash tests/run_all.sh`, or with the four psql commands in part 8.

Valid records

`tests/fixtures/valid_records.sql` loads 16 synthetic pedestrian deaths in 15 crashes. Every row is made up. The rows follow the real code lists, so they look like FARS, but none is copied from a real case; FARS describes real deaths, and a public repository is not the place for them. Population and walkability numbers are

placeholders. R1-R3, E1, and E2 are the five records of my first-pass model. Q1-Q10 are new, so that the queries in part 7 have something to rank. The whole fixture loads in one transaction and passes every constraint (log 02).

Record	Key (year, st_case)	What it represents	What it tests
R1	2021, 120417	Brevard, US-192: principal arterial, 45 mph, dark and not lighted, not at an intersection, pickup	the archetype the question expects; every link present
R2	2023, 120417	Miami-Dade: minor arterial, 35 mph, lighted intersection, marked crosswalk, left-turning sedan	same st_case as R1 in another year (Rule 1); a different countermeasure family
R3	2022, 121088	Duval: collector, 30 mph, walking along the road, rain, drinking = 9	an unknown code kept as a value (D4)
E1	2024, 120078	Orange, US-441: one SUV kills two people crossing together	one crash, two deaths, one striking vehicle (D2); 2024 Annual Report File
E2	2022, 121307	Hillsborough, SR-60: hit-and-run, no coordinates	NULL coordinates and no block group; body type 99; the death still counts in its county (Rule 3)
Q1, Q2, Q4, Q5, Q6	2020-2024	five more deaths under the archetype pattern in Brevard, Hillsborough, and Orange	pattern ranking, county concentration, 4-of-5-year stability; Q5 has age 999; Q6 has two vehicles and is struck by vehicle 2, and peds = 2 with one death
Q3, Q7, Q10	2023-2024	minor-arterial and intersection cases; Q10 in daylight	Q7 has no typing record; Q10 has no striking-vehicle link: both allowed, both read "unknown" in the view
Q8, Q9	2021, 2022	a death on I-95 (FUNC_SYS 1) and a rural death in Polk (RUR_URB 1)	outside the analysis view, inside the county rates

After loading: 15 crashes, 16 vehicles, 16 pedestrians, 15 typing records, 14 rows in v_ped_transactions (16 minus Q8 and Q9).

Invalid records and valid edge cases: 56 tests

tests/test_constraints.sql holds 56 test cases as rows of a table: the statement to send and what must come back. For an invalid record, the test names the SQLSTATE and the constraint that must reject it. A rejection by any other rule counts as a failure, so a test cannot pass by accident. For a valid edge case, the statement must be accepted. Each statement runs in its own savepoint and is rolled back; the file ends with ROLLBACK and then confirms that the fixture is unchanged. Deferred rules are fired inside the test with SET CONSTRAINTS ALL IMMEDIATE. SQLSTATE 23505 = unique violation, 23503 = foreign key, 23514 = check, 23502 = not null, 23P01 = exclusion, 428C9 = generated column, 42501 = permission denied.

Result: 56 tests, 56 passed, 0 failed. 49 invalid records were rejected by exactly the expected constraint, and 7 valid edge cases were accepted. Groups: A = keys, B = relationships, C = allowed values, D = "one or more" rules, E = access control. The last column is PostgreSQL's own error text (logs 03, as .log and .csv).

ID	Rule under test	What the test tries	Expected	What PostgreSQL did	PostgreSQL's message	Verdict
A1	Rule 1: a crash is identified by year + st_case	second crash with the key (2021, 120417)	REJECT 23505 crash_pk	REJECTED 23505 crash_pk	duplicate key value violates unique constraint "crash_pk"	PASS
A2	Rule 1: st_case is reused every year	crash (2022, 120417): same st_case as R1 and R2, new year, with its vehicle and pedestrian	ACCEPT	ACCEPTED	(accepted)	PASS
A3	a pedestrian is identified inside the crash by	second pedestrian row with the key (2024, 120078, 0, 1)	REJECT 23505 pedestrian_pk	REJECTED 23505 pedestrian_pk	duplicate key value violates unique	PASS

ID	Rule under test	What the test tries	Expected	What PostgreSQL did	PostgreSQL's message	Verdict
	per_no				constraint "pedestrian_pk"	
A4	key columns are never empty	pedestrian with per_no NULL	REJECT 23502	REJECTED 23502	null value in column "per_no" of relation "pedestrian" violates not-null constraint	PASS
A5	at most one crash-typing record per pedestrian	second ped_crash_type row for pedestrian (2021, 120417, 0, 1)	REJECT 23505 ped_crash_type_pk	REJECTED 23505 ped_crash_type_pk	duplicate key value violates unique constraint "ped_crash_type_pk"	PASS
A6	one population estimate per county and year	second estimate for Brevard 2021	REJECT 23505 county_population_pk	REJECTED 23505 county_population_pk	duplicate key value violates unique constraint "county_population_pk"	PASS
B1	Rule 1: every vehicle belongs to a crash of the same year	vehicle for crash (2020, 120001), which does not exist	REJECT 23503 vehicle_crash_fk	REJECTED 23503 vehicle_crash_fk	insert or update on table "vehicle" violates foreign key constraint "vehicle_crash_fk"	PASS
B2	Rule 1: every pedestrian belongs to a crash of the same year	pedestrian with st_case 120417 in 2020 (the case exists in 2021 and 2023 only)	REJECT 23503 pedestrian_crash_fk	REJECTED 23503 pedestrian_crash_fk	insert or update on table "pedestrian" violates foreign key constraint "pedestrian_crash_fk"	PASS
B3	Rule 2: the striking vehicle must be a vehicle of the same crash	pedestrian 2 in crash (2022, 121307) with str_veh = 2; that crash has vehicle 1 only	REJECT 23503 pedestrian_striking_vehicle_fk	REJECTED 23503 pedestrian_striking_vehicle_fk	insert or update on table "pedestrian" violates foreign key constraint "pedestrian_striking_vehicle_fk"	PASS
B4	Rule 2: "no striking vehicle" is stored as NULL, never as 0	pedestrian with str_veh = 0	REJECT 23514 pedestrian_str_veh_ck	REJECTED 23514 pedestrian_str_veh_ck	new row for relation "pedestrian" violates check constraint "pedestrian_str_veh_ck"	PASS
B5	Rule 2: the striking-vehicle link is optional	pedestrian with str_veh NULL	ACCEPT	ACCEPTED	(accepted)	PASS
B6	Rule 2: a vehicle that struck someone cannot be removed on its own	delete vehicle (2021, 120417, 1) while its pedestrian stays	REJECT 23503 pedestrian_striking_vehicle_fk	REJECTED 23503 pedestrian_striking_vehicle_fk	update or delete on table "vehicle" violates foreign key constraint "pedestrian_striking_vehicle_fk" on table "pedestrian"	PASS
B7	Rule 3: every crash has a county that exists	crash with the FARS county code 999 (unknown)	REJECT 23503 crash_county_fk	REJECTED 23503 crash_county_fk	insert or update on table "crash" violates foreign key constraint "crash_county_fk"	PASS
B8	Rule 3: a county with crashes cannot be deleted	delete Brevard (9)	REJECT 23503	REJECTED 23503 county_population_county_fk	update or delete on table "county" violates foreign key constraint "county_population_county_fk" on table "county_population"	PASS
B9	Rule 3: the block group must exist	crash geocoded to block group 120099999999	REJECT 23503 crash_block_group_fk	REJECTED 23503 crash_block_group_fk	insert or update on table "crash" violates foreign key constraint "crash_block_group_fk"	PASS

ID	Rule under test	What the test tries	Expected	What PostgreSQL did	PostgreSQL's message	Verdict
B10	a typing record needs its pedestrian	ped_crash_type row for pedestrian (2021, 120417, 0, 7), who does not exist	REJECT 23503 ped_crash_type_pedestrian_fk	REJECTED 23503 ped_crash_type_pedestrian_fk	insert or update on table "ped_crash_type" violates foreign key constraint "ped_crash_type_pedestrian_fk"	PASS
B11	the crash type must be a PBCAT pedestrian type	typing record for Q7 with pedctype 0 ("not a pedestrian")	REJECT 23503 ped_crash_type_pbcats_fk	REJECTED 23503 ped_crash_type_pbcats_fk	insert or update on table "ped_crash_type" violates foreign key constraint "ped_crash_type_pbcats_fk"	PASS
B12	a block group lies in a county that exists	block group 120250001001 (county 025, the retired Dade code)	REJECT 23503 block_group_county_fk	REJECTED 23503 block_group_county_fk	insert or update on table "block_group" violates foreign key constraint "block_group_county_fk"	PASS
B13	the county of a block group comes from its GEOID and cannot be typed in	insert a block group with county_fips written by hand	REJECT 428C9	REJECTED 428C9	cannot insert a non-DEFAULT value into column "county_fips"	PASS
B14	reload of one year: deleting a crash removes its vehicles, pedestrians, and typing records	delete every 2024 crash (the Annual Report File is replaced by the Final file)	ACCEPT	ACCEPTED	(accepted)	PASS
C1	scope: crash years 2020-2024	crash dated 2019	REJECT 23514 crash_year_ck	REJECTED 23514 crash_year_ck	new row for relation "crash" violates check constraint "crash_year_ck"	PASS
C2	scope: Florida only (st_case starts with 12)	crash with st_case 130001 (Georgia)	REJECT 23514 crash_st_case_ck	REJECTED 23514 crash_st_case_ck	new row for relation "crash" violates check constraint "crash_st_case_ck"	PASS
C3	scope: pedestrians only (PER_TYP = 5)	a bicyclist (per_typ 6) in the pedestrian table	REJECT 23514 pedestrian_per_typ_ck	REJECTED 23514 pedestrian_per_typ_ck	new row for relation "pedestrian" violates check constraint "pedestrian_per_typ_ck"	PASS
C4	scope: fatal injuries only (INJ_SEV = 4)	a pedestrian with a serious injury (inj_sev 3)	REJECT 23514 pedestrian_inj_sev_ck	REJECTED 23514 pedestrian_inj_sev_ck	new row for relation "pedestrian" violates check constraint "pedestrian_inj_sev_ck"	PASS
C5	pedestrians are outside vehicles (VEH_NO = 0)	pedestrian row with veh_no 1	REJECT 23514 pedestrian_veh_no_ck	REJECTED 23514 pedestrian_veh_no_ck	new row for relation "pedestrian" violates check constraint "pedestrian_veh_no_ck"	PASS
C6	vehicles are numbered from 1	vehicle with veh_no 0	REJECT 23514 vehicle_veh_no_ck	REJECTED 23514 vehicle_veh_no_ck	new row for relation "vehicle" violates check constraint "vehicle_veh_no_ck"	PASS
C7	LGT_COND codes are 1-9	crash with lgt_cond 0	REJECT 23514 crash_lgt_cond_ck	REJECTED 23514 crash_lgt_cond_ck	new row for relation "crash" violates check constraint "crash_lgt_cond_ck"	PASS
C8	HOUR is 0-23, or 99 for unknown	crash at hour 24	REJECT 23514 crash_hour_ck	REJECTED 23514 crash_hour_ck	new row for relation "crash" violates check constraint "crash_hour_ck"	PASS

ID	Rule under test	What the test tries	Expected	What PostgreSQL did	PostgreSQL's message	Verdict
C9	FUNC_SYS codes are 1-7, 96, 98, 99	crash with func_sys 8	REJECT 23514 crash_func_sys_ck	REJECTED 23514 crash_func_sys_ck	new row for relation "crash" violates check constraint "crash_func_sys_ck"	PASS
C10	posted speed limit is 0-95 mph, or 98/99	vehicle with vspd_lim 97	REJECT 23514 vehicle_vspd_lim_ck	REJECTED 23514 vehicle_vspd_lim_ck	new row for relation "vehicle" violates check constraint "vehicle_vspd_lim_ck"	PASS
C11	AGE is 0-120, or 998/999	pedestrian aged 130	REJECT 23514 pedestrian_age_ck	REJECTED 23514 pedestrian_age_ck	new row for relation "pedestrian" violates check constraint "pedestrian_age_ck"	PASS
C12	SEX codes are 1, 2, 8, 9	pedestrian with sex 3 (a code NHTSA withdrew from the 2022 file)	REJECT 23514 pedestrian_sex_ck	REJECTED 23514 pedestrian_sex_ck	new row for relation "pedestrian" violates check constraint "pedestrian_sex_ck"	PASS
C13	PEDLOC 7 means "not a pedestrian" and cannot occur here	typing record for Q7 with pedloc 7	REJECT 23514 ped_crash_type_pedloc_ck	REJECTED 23514 ped_crash_type_pedloc_ck	new row for relation "ped_crash_type" violates check constraint "ped_crash_type_pedloc_ck"	PASS
C14	unknown codes are values, not errors	crash with hour 99, lgt_cond 9, func_sys 99; vehicle with vspd_lim 99 and body_typ 99; pedestrian with age 998, sex 9, drinking 8	ACCEPT	ACCEPTED	(accepted)	PASS
C15	Rule 3: the FARS filler 77.7777 is not a latitude; it must arrive as NULL	crash with latitude 77.7777 and longitude 777.7777	REJECT 23514	REJECTED 23514 crash_latitude_ck	new row for relation "crash" violates check constraint "crash_latitude_ck"	PASS
C16	Rule 3: a point has both coordinates or neither	crash with a latitude and no longitude	REJECT 23514 crash_point_both_or_neither_ck	REJECTED 23514 crash_point_both_or_neither_ck	new row for relation "crash" violates check constraint "crash_point_both_or_neither_ck"	PASS
C17	Rule 3: a point must fall inside Florida's bounding box	crash at 40.7128, -74.0060 (New York)	REJECT 23514	REJECTED 23514 crash_latitude_ck	new row for relation "crash" violates check constraint "crash_latitude_ck"	PASS
C18	Rule 3: no block group without valid coordinates	crash with a block group and NULL coordinates	REJECT 23514 crash_block_group_needs_point_ck	REJECTED 23514 crash_block_group_needs_point_ck	new row for relation "crash" violates check constraint "crash_block_group_needs_point_ck"	PASS
C19	the walkability index runs from 1 to 20	block group with natwalkind 21.5	REJECT 23514 block_group_natwalkind_ck	REJECTED 23514 block_group_natwalkind_ck	new row for relation "block_group" violates check constraint "block_group_natwalkind_ck"	PASS
C20	EPA's -99999 (no transit stop nearby) must arrive as NULL	block group with d4a = -99999	REJECT 23514 block_group_d4a_ck	REJECTED 23514 block_group_d4a_ck	new row for relation "block_group" violates check constraint "block_group_d4a_ck"	PASS

ID	Rule under test	What the test tries	Expected	What PostgreSQL did	PostgreSQL's message	Verdict
C21	a block-group GEOID is 12 digits and starts with 12 (Florida)	block group 130510001001 (Georgia)	REJECT 23514 block_group_geoid_ck	REJECTED 23514 block_group_geoid_ck	new row for relation "block_group" violates check constraint "block_group_geoid_ck"	PASS
C22	a population estimate is a positive number	Lafayette 2021 with population 0	REJECT 23514 county_population_pop_ck	REJECTED 23514 county_population_pop_ck	new row for relation "county_population" violates check constraint "county_population_pop_ck"	PASS
C23	an estimate for year Y cannot come from a vintage before Y	a 2024 estimate labelled Vintage 2023	REJECT 23514 county_population_vintage_ck	REJECTED 23514 county_population_vintage_ck	new row for relation "county_population" violates check constraint "county_population_vintage_ck"	PASS
C24	one code has one label in any given year	second label for lgt_cond 2 valid 2022-2024 (the first covers 2020-2024)	REJECT 23P01 code_lookup_no_overlap_ex	REJECTED 23P01 code_lookup_no_overlap_ex	conflicting key value violates exclusion constraint "code_lookup_no_overlap_ex"	PASS
C25	a file in the manifest carries a real SHA-256 checksum	source_file row with sha256 = 'abc'	REJECT 23514 source_file_sha256_ck	REJECTED 23514 source_file_sha256_ck	new row for relation "source_file" violates check constraint "source_file_sha256_ck"	PASS
C26	a finished run has a finish time, a running one does not	etl_run marked succeeded with finished_at NULL	REJECT 23514 etl_run_finish_ck	REJECTED 23514 etl_run_finish_ck	new row for relation "etl_run" violates check constraint "etl_run_finish_ck"	PASS
D1	a crash kills one or more pedestrians	crash and vehicle, but no pedestrian row	REJECT 23514 crash_min_one_pedestrian	REJECTED 23514 crash_min_one_pedestrian	crash (2023, 120994) has no pedestrian fatality row	PASS
D2	a crash involves one or more vehicles in transport	crash and pedestrian, but no vehicle row	REJECT 23514 crash_min_one_vehicle	REJECTED 23514 crash_min_one_vehicle	crash (2023, 120994) has no in-transport vehicle row	PASS
D3	a crash kills one or more pedestrians (also after a delete)	delete the only pedestrian of crash (2021, 120417)	REJECT 23514 crash_min_one_pedestrian	REJECTED 23514 crash_min_one_pedestrian	crash (2021, 120417) has no pedestrian fatality row	PASS
D4	E1: one crash may kill several pedestrians	delete one of the two pedestrians of crash (2024, 120078); one is left, so the crash is still valid	ACCEPT	ACCEPTED	(accepted)	PASS
E1	the dashboard role cannot read person-level rows	ped_dashboard: SELECT * FROM pedestrian	REJECT 42501	REJECTED 42501	permission denied for table pedestrian	PASS
E2	the dashboard role cannot read crash coordinates	ped_dashboard: SELECT latitude, longitud FROM crash	REJECT 42501	REJECTED 42501	permission denied for table crash	PASS
E3	the dashboard role cannot read the case-level analysis view	ped_dashboard: SELECT * FROM v_ped_transactions	REJECT 42501	REJECTED 42501	permission denied for view v_ped_transactions	PASS
E4	the dashboard role can read aggregates	ped_dashboard: SELECT from v_county_rate and v_dash_pattern_county	ACCEPT	ACCEPTED	(accepted)	PASS

ID	Rule under test	What the test tries	Expected	What PostgreSQL did	PostgreSQL's message	Verdict
E5	the analyst role reads everything but changes nothing	ped_analyst: UPDATE crash SET lgt_cond = 1	REJECT 42501	REJECTED 42501	permission denied for table crash	PASS
E6	the analyst role can read the analysis view	ped_analyst: SELECT * FROM v_ped_transactions	ACCEPT	ACCEPTED	(accepted)	PASS

Two more checks

Do the CHECK lists and the code labels agree? tests/check_code_domains.py reads the CHECK constraints from sql/01_schema.sql and the labels from sql/07_seed_codes.sql and compares them code by code (log 05):

```
PASS  rur_urb    5 codes accepted,  5 labelled
PASS  func_sys  10 codes accepted, 10 labelled
PASS  reljct2   15 codes accepted, 15 labelled
PASS  lgt_cond   9 codes accepted,  9 labelled
PASS  weather  13 codes accepted, 13 labelled
PASS  day_week   7 codes accepted,  7 labelled
PASS  hit_run    3 codes accepted,  3 labelled
PASS  per_typ    1 codes accepted,  1 labelled
PASS  inj_sev    1 codes accepted,  1 labelled
PASS  sex        4 codes accepted,  4 labelled
PASS  drinking  4 codes accepted,  4 labelled
PASS  pbcwalk   3 codes accepted,  3 labelled
PASS  pbswalk   3 codes accepted,  3 labelled
PASS  pedloc    5 codes accepted,  5 labelled
PASS  pedpos    9 codes accepted,  9 labelled
PASS  pbcats    55 crash types in 16 crash groups
```

CHECK constraints and code labels agree.

Do the data-quality views find what they should? On the fixture they report exactly the cases I planted (log 04): Q7 without a typing record, Q10 without a vehicle link, and 300 missing county-years (the fixture has population for 7 of 67 counties). The four views that must be empty are empty, including v_dq_undocumented_column.

check_view	rows_found	reading
v_dq_count_mismatch	0	must be 0
v_dq_county_block_group_mismatch	0	must be 0 or explained
v_dq_unlabeled_code	0	must be 0
v_dq_undocumented_column	0	must be 0
v_dq_missing_typing	1	allowed; Q7 has none
v_dq_no_striking_vehicle	1	allowed; Q10 has none
v_dq_population_gap	300	300 = 60 counties x 5 years not in the fixture

7. Sample Queries Tied to the Project Question

The question has two halves: which combinations of conditions are most frequent, and which counties concentrate the deaths under each. Q1 and Q4 serve the first half, Q2 and Q3 the second. Q5 feeds the dashboard heatmap. Q6 checks that the data can carry the answer at all. The file is `queries/sample_queries.sql`; the full output is log 04.

The results below come from the 16 synthetic rows. They are not findings. I built the fixture so that one pattern dominates, to check that the queries rank, join, and count the edge cases correctly.

Q1. Which combinations of conditions are most frequent? (question, part a)

```
SELECT road_class, speed_bin, light, crossing_location, time_of_day,
       count(*) AS deaths,
       round(100.0 * count(*) / sum(count(*) OVER (), 1) AS support_pct
FROM v_ped_transactions
GROUP BY road_class, speed_bin, light, crossing_location, time_of_day
ORDER BY deaths DESC, road_class, speed_bin, time_of_day
LIMIT 10;
```

road_class	speed_bin	light	crossing_location	time_of_day	deaths	support_pct
principal arterial	45-50 mph	dark, not lighted	not at intersection	night (20-23)	5	35.7
principal arterial	45-50 mph	dark, lighted	not at intersection	night (20-23)	2	14.3
collector	30 mph or less	dark, not lighted	not at intersection	late night (0-5)	1	7.1
minor arterial	35-40 mph	dark, lighted	at intersection	evening (16-19)	1	7.1
minor arterial	45-50 mph	dark, not lighted	not at intersection	evening (16-19)	1	7.1
minor arterial	unknown	daylight	at intersection	midday (10-15)	1	7.1
principal arterial	35-40 mph	dark, not lighted	not at intersection	late night (0-5)	1	7.1
principal arterial	35-40 mph	dark, lighted	unknown	night (20-23)	1	7.1
principal arterial	45-50 mph	dark, not lighted	not at intersection	late night (0-5)	1	7.1

Reading: the archetype (principal arterial, 45-50 mph, dark and not lighted, not at an intersection, at night) comes out on top with 5 of 14 in-scope deaths. The row with speed "unknown" is Q10, which has no striking-vehicle link, and the row with crossing location "unknown" is Q7, which has no typing record. Both deaths are still counted, which is the point of D4. This GROUP BY is also the baseline that the FP-Growth rules must beat.

Q2. Which counties concentrate the most deaths under one pattern? (question, part b)

```
WITH pattern AS (
  SELECT county_fips, county_name, count(*) AS pattern_deaths
  FROM v_ped_transactions
  WHERE road_class = 'principal arterial'
        AND speed_bin = '45-50 mph'
        AND light = 'dark, not lighted'
        AND crossing_location = 'not at intersection'
  GROUP BY county_fips, county_name
)
SELECT p.county_name,
       p.pattern_deaths,
       r.deaths_in_scope,
       round(100.0 * p.pattern_deaths / r.deaths_in_scope, 1) AS share_of_county_pct,
       round(100000.0 * p.pattern_deaths / r.person_years, 3) AS pattern_rate_per_100k_per_year
FROM pattern p
```

```
JOIN v_county_rate r ON r.county_fips = p.county_fips
ORDER BY p.pattern_deaths DESC, p.county_name;
```

county_name	pattern_deaths	deaths_in_scope	share_of_county_pct	pattern_rate_per_100k_per_year
Brevard	3	4	75.0	0.096
Hillsborough	2	3	66.7	0.026
Orange	1	3	33.3	0.014

Reading: Brevard concentrates the pattern (3 of its 4 in-scope deaths). Dividing by person-years from v_county_rate keeps a large county from winning on size alone.

Q3. Five-year county fatality rates per 100,000 residents (Objective 3)

```
(SELECT county_name, deaths_all, deaths_in_scope, person_years, rate_per_100k_per_year, unstable_rate
FROM v_county_rate
WHERE deaths_all > 0
ORDER BY rate_per_100k_per_year DESC, county_name
LIMIT 10)
UNION ALL
(SELECT county_name, deaths_all, deaths_in_scope, person_years, rate_per_100k_per_year, unstable_rate
FROM v_county_rate
WHERE county_name = 'Lafayette');
```

county_name	deaths_all	deaths_in_scope	person_years	rate_per_100k_per_year	unstable_rate
Brevard	4	4	3130000	0.13	t
Duval	2	1	5086000	0.04	t
Hillsborough	3	3	7564000	0.04	t
Orange	3	3	7298000	0.04	t
Polk	1	0	3935000	0.03	t
Miami-Dade	3	3	13546000	0.02	t
Lafayette	0	0	41726	0.00	t

Reading: every rate is flagged unstable, as it should be with fewer than 20 deaths. Duval shows 2 deaths but 1 in scope (Q8 died on I-95), and Polk shows 1 and 0 (Q9 was rural): the rate counts them, the pattern view does not. Lafayette has no deaths and still appears, with rate 0.

Q4. Does the pattern hold year after year? (Objective 2: stable in 4 of 5 years)

```
SELECT y.year,
       count(t.st_case) AS pattern_deaths,
       (SELECT count(*) FROM v_ped_transactions a WHERE a.year = y.year) AS all_in_scope_deaths
FROM generate_series(2020, 2024) AS y(year)
LEFT JOIN v_ped_transactions t
      ON t.year = y.year
      AND t.road_class = 'principal arterial'
      AND t.speed_bin = '45-50 mph'
      AND t.light = 'dark, not lighted'
      AND t.crossing_location = 'not at intersection'
GROUP BY y.year
ORDER BY y.year;
```

year	pattern_deaths	all_in_scope_deaths
2020	2	2
2021	2	2
2022	1	3
2023	0	3
2024	1	4

Reading: the pattern appears in 2020, 2021, 2022, and 2024, which is 4 of 5 years, the stability threshold of Objective 2. The year 2023 shows 0 instead of vanishing, because the years come from generate_series and not from the data.

Q5. Hour-by-lighting matrix for the dashboard heatmap

```
SELECT time_of_day,
       count(*) FILTER (WHERE light = 'daylight')           AS daylight,
       count(*) FILTER (WHERE light = 'dawn or dusk')       AS dawn_dusk,
       count(*) FILTER (WHERE light = 'dark, lighted')      AS dark_lighted,
       count(*) FILTER (WHERE light = 'dark, not lighted')  AS dark_not_lighted,
       count(*) FILTER (WHERE light IN ('dark, lighting unknown', 'unknown')) AS unknown,
       count(*)                                             AS total
FROM v_ped_transactions
GROUP BY time_of_day
ORDER BY min(CASE time_of_day WHEN 'late night (0-5)' THEN 1 WHEN 'morning (6-9)' THEN 2
                        WHEN 'midday (10-15)' THEN 3 WHEN 'evening (16-19)' THEN 4
                        WHEN 'night (20-23)' THEN 5 ELSE 6 END);
```

time_of_day	daylight	dawn_dusk	dark_lighted	dark_not_lighted	unknown	total
late night (0-5)	0	0	0	3	0	3
midday (10-15)	1	0	0	0	0	1
evening (16-19)	0	0	1	1	0	2
night (20-23)	0	0	3	5	0	8

Reading: deaths cluster at night and late at night on unlighted roads. The matrix is the data behind the dashboard's hour-by-lighting heatmap.

Q6. Can the data carry the answer? Yearly counts and coverage (Objective 1)

```
SELECT c.year,
       count(*)                                             AS deaths_all,
       count(*) FILTER (WHERE c.rur_urb = 2 AND c.func_sys NOT IN (1, 2)) AS deaths_in_scope,
       count(*) FILTER (WHERE c.latitude IS NOT NULL)      AS geocoded,
       round(100.0 * count(*) FILTER (WHERE c.geoid10 IS NOT NULL)
            / NULLIF(count(*) FILTER (WHERE c.latitude IS NOT NULL), 0), 1) AS
walkability_pct_of_geocoded,
       count(*) FILTER (WHERE p.str_veh IS NULL)           AS no_striking_vehicle,
       count(*) FILTER (WHERE v.vspd_lim IN (98, 99))     AS speed_limit_unknown,
       count(*) FILTER (WHERE t.per_no IS NULL)           AS no_typing_record
FROM pedestrian p
JOIN crash c ON c.year = p.year AND c.st_case = p.st_case
LEFT JOIN vehicle v ON v.year = p.year AND v.st_case = p.st_case AND v.veh_no = p.str_veh
LEFT JOIN ped_crash_type t ON t.year = p.year AND t.st_case = p.st_case AND t.veh_no = p.veh_no AND t.per_no =
p.per_no
GROUP BY c.year
```

ORDER BY c.year;

year	deaths_all	deaths_in_scope	geocoded	walkability_pct_of_geocoded	no_striking_vehicle	speed_limit_unknown	no_typing_record
2020	2	2	2	100.0	0	0	0
2021	3	2	3	100.0	0	0	0
2022	4	3	3	100.0	0	0	0
2023	3	3	3	100.0	0	0	1
2024	4	4	4	100.0	1	0	0

Reading: deaths_all is the number to reconcile with NHTSA's published Florida totals (Objective 1: within 1%). Crash E2 (2022) has no coordinates, so 2022 shows 3 geocoded of 4. Walkability coverage is measured against geocoded deaths, as Objective 1 defines it.

8. Design Rationale, Known Limitations, and Setup

Design rationale

The question asks about combinations, so the design's first job is to bring five conditions onto one row per death without losing or doubling anyone. FARS records those conditions in three places. Lighting, road class, junction, and hour describe the scene, so they belong to the crash. The posted speed limit belongs to the road the striking vehicle was on, and FARS records it per vehicle. Crossing location belongs to the pedestrian's crash-typing record. I kept each value where FARS records it and let keys carry the joins. That is why the unit of observation, the pedestrian, is a child of the crash and not a column group inside it: a crash can kill two people under the same conditions, and planners count deaths.

The second job is the "where". Every death must land in a county, because a county rate that skips deaths with bad coordinates would be biased. So county is mandatory, and block group is optional. The denominators sit in their own table, one row per county and year.

The third job is trust. I wanted a wrong file to fail at the door instead of bending the results later. So every coded column has a CHECK written from the FARS manuals, the scope (Florida, 2020-2024, pedestrians, fatal) is pinned by constraints, and every constraint has a name that says which rule broke. "Unknown" stays a legal value, because a planner should see that the speed limit is unknown in some share of cases. NULL is reserved for values the source does not have at all, and the dictionary says what each NULL means.

The last job is care for the people in the data. FARS is public, but these are real deaths. Race and ethnicity are never loaded, the exact date is not loaded, and the dashboard role can read aggregates only. The rule from my proposal, that person-level rows stay in the local database, is now a permission and not just a promise.

Known limitations

- **No real data yet.** The CHECK lists come from the manuals, not from profiling the files. The first real load may hit values I did not expect, for example a speed limit outside 0-95 or a blank trafficway name. That is the intended behavior (the load stops and names the rule), but some constraints may need a documented adjustment.
- **The striking-vehicle link is unverified on real rows.** By the manual's definition STR_VEH is the number of a vehicle in transport, so it should always match a VEHICLE row. I have not confirmed

that for 2020-2024. Parked and working vehicles (the PARKWORK file) are not loaded. A pedestrian whose STR_VEH matches nothing gets a NULL link, and the pipeline logs how many.

- **"One or more" depends on triggers, and only for crashes.** The triggers fire at COMMIT, so a load must put a crash and its children in one transaction. A superuser can disable triggers; `v_dq_count_mismatch` and the pipeline checks are the second line of defense. The other two "one or more" ends in the diagram concern reference data and are not enforced: a county's yearly estimates are checked by `v_dq_population_gap`, and its block groups by the pipeline's row counts.
- **Labels are not tied to the data by foreign keys,** except for PBCAT. The CHECK lists and `code_lookup` are compared by a script, not by the database. BODY_TYP labels (about 70 codes) are not seeded yet; the column is guarded by a range CHECK until then.
- **Two sources can disagree on the county.** FARS codes a county, and the coordinates fall in a block group of some county. A mismatch is listed by a view, not rejected, because either one can be wrong.
- **One Census vintage at a time.** Reloading population replaces the old estimates. The vintage is recorded, but earlier vintages are not kept.
- **2010 geography.** The EPA index uses 2010-vintage block groups (GEOID10). That is consistent inside the project, but these are not the 2020 boundaries.
- **The scope is hard-coded.** Adding 2025 or another state needs a migration that changes the CHECKs. I chose that on purpose: the scope should not grow silently.
- **Roles guard against accidents, not against a determined user.** The database is local, the superuser sees everything, and there is no row-level security.
- **Where the evidence was produced.** The logs in `docs/evidence/` come from PostgreSQL 16.13 on Linux (see the AI-assistance note). The scripts use only standard PostgreSQL and psql features, but I have not yet repeated the run on my Windows machine.

Setup and execution

Requirements: PostgreSQL 16 with psql, and Git. Python 3 is needed only for the optional code-domain check. No passwords, tokens, or keys are stored anywhere in the repository; `.gitignore` keeps `.env` files, raw data, and database dumps out.

Windows.

1. Install PostgreSQL 16 with the installer from [postgresql.org/download/windows](https://www.postgresql.org/download/windows). Keep the default components. The installer asks for a password for the postgres user: choose your own. It is typed at the prompt and never saved in a project file.
2. Open a terminal where psql is available (for example after adding `C:\Program Files\PostgreSQL\16\bin` to PATH).
3. Get the code: `git clone https://github.com/Sorlna/florida-pedestrian-patterns.git` and then `cd florida-pedestrian-patterns`. All commands below run from this folder, because the scripts read `docs/data_dictionary.csv` by a relative path.
4. Create the database and build it:

```
createdb -U postgres ped_safety
psql -U postgres -d ped_safety -v ON_ERROR_STOP=1 -f sql/00_build_all.sql
```

5. Load the synthetic test records, run the constraint tests, run the queries:

```
psql -U postgres -d ped_safety -v ON_ERROR_STOP=1 -f tests/fixtures/valid_records.sql
psql -U postgres -d ped_safety -f tests/test_constraints.sql
psql -U postgres -d ped_safety -f queries/sample_queries.sql
```

6. Optional: `python tests/check_code_domains.py`, and `psql -U postgres -d ped_safety -f tools/export_data_dictionary.sql` to regenerate `docs/data_dictionary.md`.

macOS or Linux. Install with `brew install postgresql@16` and `brew services start postgresql@16` (or the distribution's package). Then create `db ped_safety` and `bash tests/run_all.sh`. The script runs all the steps above and rewrites the logs in `docs/evidence/`.

What to expect. The build prints 15 tables, 10 views, 28 indexes. The test file ends with 56 tests, 56 passed, 0 failed and a line confirming that the fixture is untouched. Rebuilding is safe to repeat: `00_build_all.sql` drops and recreates the schema, so never point it at a database that holds anything else.

A personal login (optional, for working as an analyst rather than as the superuser). In `psql`: `CREATE ROLE your_name LOGIN;` then `\password your_name` (typed, not stored), then `GRANT ped_analyst TO your_name;`.

Files in this submission

File	Content
<code>Week5_Database_Design_Shkirpan.pdf / .docx</code>	this document (also in <code>docs/</code>)
<code>sql/00_build_all.sql ... 08_load_data_dictionary.sql</code>	the SQL definition scripts (part 4)
<code>docs/data_dictionary.csv, docs/data_dictionary.md</code>	the complete data dictionary (part 5)
<code>docs/erd_conceptual.*, docs/erd_logical.*, docs/erd_logical_pipeline.*</code>	diagram sources (.mmd) and renderings (.svg, .png)
<code>tests/fixtures/valid_records.sql, tests/test_constraints.sql, tests/check_code_domains.py, tests/run_all.sh</code>	valid records, the 56 tests, the code-list check, the one-command runner (part 6)
<code>queries/sample_queries.sql</code>	the six sample queries (part 7)
<code>docs/evidence/01 ... 05</code>	logs of the build, the fixture load, the tests (.log and .csv), the queries, and the code-list check

References

Anthropic. (2026). *Claude* [Large language model]. <https://claude.ai>

National Highway Traffic Safety Administration. (2024). *Fatality Analysis Reporting System analytical user's manual, 1975-2022* (Report No. DOT HS 813 556). U.S. Department of Transportation. <https://rosap.nhtl.bts.gov/view/dot/78035>

National Highway Traffic Safety Administration. (2025a). *Fatality Analysis Reporting System analytical user's manual, 1975-2023* (Report No. DOT HS 813 706). U.S. Department of Transportation. <https://crashstats.nhtsa.dot.gov/Api/Public/ViewPublication/813706>

National Highway Traffic Safety Administration. (2025b). *2023 FARS/CRSS pedestrian bicyclist crash typing manual* (Report No. DOT HS 813 694). U.S. Department of Transportation. <https://crashstats.nhtsa.dot.gov/Api/Public/ViewPublication/813694>

National Highway Traffic Safety Administration. (2026). *Fatality Analysis Reporting System (FARS): 2020-2024 national CSV files and release notes* [Data set]. U.S. Department of Transportation. <https://www.nhtsa.gov/file-downloads?p=nhtsa/downloads/FARS/>

- PostgreSQL Global Development Group. (2023). *PostgreSQL 16 documentation*.
<https://www.postgresql.org/docs/16/>
- Shkirpan, Y. (2026). *Project proposal: Built for cars, deadly for walkers* [Unpublished course assignment, Week 3]. Eastern Florida State College.
- Tan, P.-N., Steinbach, M., Karpatne, A., & Kumar, V. (2019). *Introduction to data mining* (2nd ed.). Pearson.
- U.S. Census Bureau. (2026). *County population totals and components of change: 2020-2025 (Vintage 2025)* [Data set]. <https://www.census.gov/data/tables/time-series/demo/popest/2020s-counties-total.html>
- U.S. Environmental Protection Agency. (2021a). *National Walkability Index: Methodology and user guide*.
https://www.epa.gov/sites/default/files/2021-06/documents/national_walkability_index_methodology_and_user_guide_june2021.pdf
- U.S. Environmental Protection Agency. (2021b). *Walkability index (Smart Location Database v3)* [Data set]. Data.gov. <https://catalog.data.gov/dataset/walkability-index>
- U.S. Environmental Protection Agency. (2021c). *Smart Location Database technical documentation and user guide, version 3.0*.
https://www.epa.gov/system/files/documents/2023-10/epa_sld_3.0_technicaldocumentationuser_guide_may2021_0.pdf

Figures 1 and 2 are my own, drawn from the Mermaid sources in docs/ (Mermaid CLI 11). All test data is synthetic; no FARS record is reproduced.

AI-assistance statement

I used Claude (Anthropic, 2026), an AI assistant, in preparing this submission. It helped me draft the SQL scripts, the test harness, the data dictionary entries, and the wording of this document, and it checked the FARS and EPA code lists against the manuals cited above. It also ran the scripts on PostgreSQL 16.13 in its own Linux environment; the logs in docs/evidence/ come from that run. The project question, scope, sources, business rules, and test records R1-E2 come from my approved proposal and my first-pass conceptual model. I reviewed the generated material and I am responsible for the final content.

Appendix A. SQL Definition Script

This is `sql/01_schema.sql` followed by `sql/02_cardinality_triggers.sql`, exactly as in the repository. Indexes, views, roles, and seeds are in the other files of `sql/`.

`sql/01_schema.sql`

```
-- =====
-- Built for Cars, Deadly for Walkers: Florida pedestrian deaths, 2020-2024
-- 01_schema.sql  tables, keys, and column rules
-- Target: PostgreSQL 16 (uses the btree_gist extension that ships with it)
-- Run through sql/00_build_all.sql, not on its own.
--
-- Conventions
-- * Column names are the FARS / EPA / Census names in lower case
--   (so "longitud" is spelled the FARS way on purpose).
-- * Every coded FARS column is NOT NULL. "Unknown" and "not reported" are
--   real codes (8, 9, 98, 99, 998, 999) and stay in the column.
-- * NULL is used only where the source has no usable value at all
--   (coordinates, block group, striking vehicle, transit distance).
--   The data dictionary says what NULL means for each of these.
-- * Every constraint has a name, so a failed load says which rule broke.
-- =====

CREATE EXTENSION IF NOT EXISTS btree_gist;  -- needed for one EXCLUDE constraint (code_lookup)

-- -----
-- Reference data
-- -----

CREATE TABLE county (
    county_fips smallint NOT NULL,
    name        text      NOT NULL,
    CONSTRAINT county_pk      PRIMARY KEY (county_fips),
    CONSTRAINT county_name_uq UNIQUE (name),
    CONSTRAINT county_fips_ck CHECK (county_fips BETWEEN 1 AND 133),
    CONSTRAINT county_name_ck CHECK (name = btrim(name) AND name <> '')
);

CREATE TABLE county_population (
    county_fips smallint NOT NULL,
    year        smallint NOT NULL,
    population   integer  NOT NULL,
    vintage     smallint NOT NULL,
    CONSTRAINT county_population_pk      PRIMARY KEY (county_fips, year),
    CONSTRAINT county_population_county_fk FOREIGN KEY (county_fips) REFERENCES county (county_fips),
    CONSTRAINT county_population_year_ck CHECK (year BETWEEN 2020 AND 2024),
    CONSTRAINT county_population_pop_ck  CHECK (population > 0),
    -- an estimate for July 1 of year Y first appears in Vintage Y
    CONSTRAINT county_population_vintage_ck CHECK (vintage BETWEEN year AND 2030)
);

CREATE TABLE block_group (
    geoid10     text NOT NULL,
    -- digits 3-5 of the GEOID are the county, so the column is computed, not typed in.
    -- It cannot disagree with the GEOID, and it still carries a foreign key.
    county_fips smallint NOT NULL GENERATED ALWAYS AS (substring(geoid10 FROM 3 FOR 3)::smallint) STORED,
    natwalkind  numeric(5,2) NOT NULL,
    d3b        numeric(9,3),
```

```

d4a         numeric(9,2),
totpop      integer NOT NULL,
CONSTRAINT block_group_pk          PRIMARY KEY (geoid10),
CONSTRAINT block_group_county_fk   FOREIGN KEY (county_fips) REFERENCES county (county_fips),
CONSTRAINT block_group_geoid_ck    CHECK (geoid10 ~ '^12[0-9]{10}$'),      -- 12 digits, Florida
CONSTRAINT block_group_natwalkind_ck CHECK (natwalkind BETWEEN 1 AND 20),
CONSTRAINT block_group_d3b_ck      CHECK (d3b >= 0),
CONSTRAINT block_group_d4a_ck      CHECK (d4a >= 0),                      -- EPA's -99999 must arrive as NULL
CONSTRAINT block_group_totpop_ck    CHECK (totpop >= 0)
);

-- Flat FARS code lists: labels by variable, code, and the years the label is valid.
-- Allowed values themselves are enforced by CHECK constraints on the data tables;
-- this table supplies the words for views, the dashboard, and the dictionary.
CREATE TABLE code_lookup (
    variable text NOT NULL,
    code     smallint NOT NULL,
    year_from smallint NOT NULL DEFAULT 2020,
    year_to   smallint NOT NULL DEFAULT 2024,
    label     text NOT NULL,
    is_unknown boolean NOT NULL DEFAULT false,
    CONSTRAINT code_lookup_pk          PRIMARY KEY (variable, code, year_from),
    CONSTRAINT code_lookup_variable_ck CHECK (variable = lower(variable) AND variable <> ''),
    CONSTRAINT code_lookup_years_ck    CHECK (year_from BETWEEN 2020 AND 2024 AND year_to BETWEEN year_from AND 2024),
    CONSTRAINT code_lookup_label_ck    CHECK (label <> ''),
    -- one code cannot have two labels in the same year
    CONSTRAINT code_lookup_no_overlap_ex EXCLUDE USING gist
        (variable WITH =, code WITH =, int4range(year_from, year_to, '[') WITH &&)
);

-- Pedestrian crash typing (PBCAT) has a hierarchy: every crash type belongs to one
-- crash group. That is a real dependency, so it gets real tables and foreign keys.
CREATE TABLE pbcats_group (
    pedcgp smallint NOT NULL,
    label text NOT NULL,
    CONSTRAINT pbcats_group_pk          PRIMARY KEY (pedcgp),
    CONSTRAINT pbcats_group_label_uq    UNIQUE (label)
);

CREATE TABLE pbcats_type (
    pedctype smallint NOT NULL,
    label text NOT NULL,
    pedcgp smallint NOT NULL,
    CONSTRAINT pbcats_type_pk          PRIMARY KEY (pedctype),
    CONSTRAINT pbcats_type_label_uq    UNIQUE (label),
    CONSTRAINT pbcats_type_group_fk    FOREIGN KEY (pedcgp) REFERENCES pbcats_group (pedcgp)
);

-- -----
-- FARS data: crash -> vehicle, crash -> pedestrian -> ped_crash_type
-- -----

CREATE TABLE crash (
    year     smallint NOT NULL,
    st_case  integer NOT NULL,
    county   smallint NOT NULL,
    month    smallint NOT NULL,
    day_week smallint NOT NULL,
    hour     smallint NOT NULL,

```

```

    rur_urb    smallint NOT NULL,
    func_sys   smallint NOT NULL,
    reljct2    smallint NOT NULL,
    lgt_cond   smallint NOT NULL,
    weather    smallint NOT NULL,
    latitude   numeric(10,8),
    longitud   numeric(11,8),
    geoid10    text,
    tway_id    text,
    fatals     smallint NOT NULL,
    peds       smallint NOT NULL,
    CONSTRAINT crash_pk          PRIMARY KEY (year, st_case),
    CONSTRAINT crash_county_fk   FOREIGN KEY (county) REFERENCES county (county_fips),
    CONSTRAINT crash_block_group_fk FOREIGN KEY (geoid10) REFERENCES block_group (geoid10),
    CONSTRAINT crash_year_ck     CHECK (year BETWEEN 2020 AND 2024),
    CONSTRAINT crash_st_case_ck  CHECK (st_case BETWEEN 120001 AND 129999),    -- first two digits = state 12, Florida
    CONSTRAINT crash_month_ck    CHECK (month BETWEEN 1 AND 12),
    CONSTRAINT crash_day_week_ck CHECK (day_week BETWEEN 1 AND 7),
    CONSTRAINT crash_hour_ck     CHECK (hour BETWEEN 0 AND 23 OR hour = 99),
    CONSTRAINT crash_rur_urb_ck  CHECK (rur_urb IN (1, 2, 6, 8, 9)),
    CONSTRAINT crash_func_sys_ck CHECK (func_sys IN (1, 2, 3, 4, 5, 6, 7, 96, 98, 99)),
    CONSTRAINT crash_reljct2_ck  CHECK (reljct2 IN (1, 2, 3, 4, 5, 6, 7, 8, 16, 17, 18, 19, 20, 98, 99)),
    CONSTRAINT crash_lgt_cond_ck CHECK (lgt_cond BETWEEN 1 AND 9),
    CONSTRAINT crash_weather_ck  CHECK (weather IN (1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 98, 99)),
    -- a point has both parts or neither; the FARS fillers 77.7777 / 777.7777 etc. fall outside the box
    CONSTRAINT crash_point_both_or_neither_ck CHECK ((latitude IS NULL) = (longitud IS NULL)),
    CONSTRAINT crash_latitude_ck  CHECK (latitude BETWEEN 24.3 AND 31.1),
    CONSTRAINT crash_longitud_ck  CHECK (longitud BETWEEN -87.7 AND -79.8),
    -- business rule 3: a block group only when there is a point to put in it
    CONSTRAINT crash_block_group_needs_point_ck CHECK (geoid10 IS NULL OR latitude IS NOT NULL),
    CONSTRAINT crash_tway_id_ck   CHECK (tway_id = btrim(tway_id) AND tway_id <> ''),
    CONSTRAINT crash_fatals_ck    CHECK (fatals BETWEEN 1 AND 99),
    CONSTRAINT crash_peds_ck      CHECK (peds BETWEEN 1 AND 99)
);

CREATE TABLE vehicle (
    year      smallint NOT NULL,
    st_case   integer  NOT NULL,
    veh_no    smallint NOT NULL,
    body_typ  smallint NOT NULL,
    vspd_lim  smallint NOT NULL,
    hit_run   smallint NOT NULL,
    CONSTRAINT vehicle_pk          PRIMARY KEY (year, st_case, veh_no),
    CONSTRAINT vehicle_crash_fk    FOREIGN KEY (year, st_case) REFERENCES crash (year, st_case) ON DELETE CASCADE,
    CONSTRAINT vehicle_veh_no_ck   CHECK (veh_no BETWEEN 1 AND 999),          -- 0 is reserved for people outside vehicles
    CONSTRAINT vehicle_body_typ_ck CHECK (body_typ BETWEEN 1 AND 99),
    CONSTRAINT vehicle_vspd_lim_ck CHECK (vspd_lim BETWEEN 0 AND 95 OR vspd_lim IN (98, 99)),
    CONSTRAINT vehicle_hit_run_ck  CHECK (hit_run IN (0, 1, 9))
);

CREATE TABLE pedestrian (
    year      smallint NOT NULL,
    st_case   integer  NOT NULL,
    veh_no    smallint NOT NULL,
    per_no    smallint NOT NULL,
    str_veh   smallint,
    per_typ   smallint NOT NULL,
    inj_sev   smallint NOT NULL,
    age       smallint NOT NULL,

```

```

sex          smallint NOT NULL,
drinking smallint NOT NULL,
CONSTRAINT pedestrian_pk          PRIMARY KEY (year, st_case, veh_no, per_no),
CONSTRAINT pedestrian_crash_fk FOREIGN KEY (year, st_case) REFERENCES crash (year, st_case) ON DELETE CASCADE,
-- business rules 1 and 2: the striking vehicle must sit in the same crash of the same year
CONSTRAINT pedestrian_striking_vehicle_fk FOREIGN KEY (year, st_case, str_veh)
    REFERENCES vehicle (year, st_case, veh_no),
CONSTRAINT pedestrian_veh_no_ck    CHECK (veh_no = 0),                -- people outside vehicles
CONSTRAINT pedestrian_per_no_ck    CHECK (per_no BETWEEN 1 AND 999),
CONSTRAINT pedestrian_str_veh_ck    CHECK (str_veh BETWEEN 1 AND 999),    -- "no link" is NULL, never 0
CONSTRAINT pedestrian_per_typ_ck    CHECK (per_typ = 5),                -- scope: pedestrians only
CONSTRAINT pedestrian_inj_sev_ck    CHECK (inj_sev = 4),                -- scope: fatal injuries only
CONSTRAINT pedestrian_age_ck        CHECK (age BETWEEN 0 AND 120 OR age IN (998, 999)),
CONSTRAINT pedestrian_sex_ck        CHECK (sex IN (1, 2, 8, 9)),
CONSTRAINT pedestrian_drinking_ck   CHECK (drinking IN (0, 1, 8, 9))
);

CREATE TABLE ped_crash_type (
    year          smallint NOT NULL,
    st_case       integer  NOT NULL,
    veh_no        smallint NOT NULL,
    per_no        smallint NOT NULL,
    pbcwalk       smallint NOT NULL,
    pbswalk       smallint NOT NULL,
    pedloc        smallint NOT NULL,
    pedpos        smallint NOT NULL,
    pedctype      smallint NOT NULL,
    CONSTRAINT ped_crash_type_pk      PRIMARY KEY (year, st_case, veh_no, per_no),    -- same key = at most one row per
pedestrian
    CONSTRAINT ped_crash_type_pedestrian_fk FOREIGN KEY (year, st_case, veh_no, per_no)
        REFERENCES pedestrian (year, st_case, veh_no, per_no) ON DELETE CASCADE,
    CONSTRAINT ped_crash_type_pbcat_fk    FOREIGN KEY (pedctype) REFERENCES pbcat_type (pedctype),
    CONSTRAINT ped_crash_type_pbcwalk_ck  CHECK (pbcwalk IN (0, 1, 9)),
    CONSTRAINT ped_crash_type_pbswalk_ck  CHECK (pbswalk IN (0, 1, 9)),
    CONSTRAINT ped_crash_type_pedloc_ck    CHECK (pedloc IN (1, 2, 3, 4, 9)),    -- 7 "not a pedestrian" cannot occur here
    CONSTRAINT ped_crash_type_pedpos_ck    CHECK (pedpos BETWEEN 1 AND 9)        -- 77 "not a pedestrian" cannot occur here
);

-- Pipeline bookkeeping (from the proposal: manifest, etl_log, validation_result)
-- -----

CREATE TABLE etl_run (
    run_id        integer      GENERATED ALWAYS AS IDENTITY,
    started_at    timestamptz NOT NULL DEFAULT now(),
    finished_at   timestamptz,
    git_commit    text,
    status        text         NOT NULL DEFAULT 'running',
    CONSTRAINT etl_run_pk          PRIMARY KEY (run_id),
    CONSTRAINT etl_run_status_ck   CHECK (status IN ('running', 'succeeded', 'failed')),
    CONSTRAINT etl_run_commit_ck   CHECK (git_commit ~ '^([0-9a-f]){7,40}$'),
    CONSTRAINT etl_run_finish_ck   CHECK ((status = 'running') = (finished_at IS NULL)),
    CONSTRAINT etl_run_order_ck    CHECK (finished_at >= started_at)
);

CREATE TABLE source_file (
    file_id       integer      GENERATED ALWAYS AS IDENTITY,
    run_id        integer      NOT NULL,
    source        text         NOT NULL,

```

```

data_year    smallint,
release      text    NOT NULL,
file_name    text    NOT NULL,
url          text    NOT NULL,
sha256       text    NOT NULL,
retrieved_on date    NOT NULL,
CONSTRAINT source_file_pk      PRIMARY KEY (file_id),
CONSTRAINT source_file_run_fk  FOREIGN KEY (run_id) REFERENCES etl_run (run_id) ON DELETE CASCADE,
CONSTRAINT source_file_name_uq UNIQUE (run_id, file_name),
CONSTRAINT source_file_source_ck CHECK (source IN ('FARS', 'EPA_SLD', 'CENSUS_POPEST')),
-- FARS files are yearly; the EPA and Census files are not
CONSTRAINT source_file_year_ck CHECK ((source = 'FARS') = (data_year IS NOT NULL) AND data_year BETWEEN 2020 AND 2024),
CONSTRAINT source_file_url_ck  CHECK (url ~ '^https://'),
CONSTRAINT source_file_sha256_ck CHECK (sha256 ~ '^[0-9a-f]{64}$')
);

CREATE TABLE etl_log (
log_id       integer    GENERATED ALWAYS AS IDENTITY,
run_id       integer    NOT NULL,
step         text       NOT NULL,
rows_in      integer,
rows_out     integer,
status       text       NOT NULL,
message      text,
logged_at    timestamptz NOT NULL DEFAULT now(),
CONSTRAINT etl_log_pk        PRIMARY KEY (log_id),
CONSTRAINT etl_log_run_fk    FOREIGN KEY (run_id) REFERENCES etl_run (run_id) ON DELETE CASCADE,
CONSTRAINT etl_log_status_ck CHECK (status IN ('ok', 'warning', 'error')),
CONSTRAINT etl_log_rows_ck   CHECK (rows_in >= 0 AND rows_out >= 0)
);

CREATE TABLE validation_result (
result_id    integer    GENERATED ALWAYS AS IDENTITY,
run_id       integer    NOT NULL,
check_name   text       NOT NULL,
passed       boolean     NOT NULL,
detail       text,
checked_at   timestamptz NOT NULL DEFAULT now(),
CONSTRAINT validation_result_pk PRIMARY KEY (result_id),
CONSTRAINT validation_result_run_fk FOREIGN KEY (run_id) REFERENCES etl_run (run_id) ON DELETE CASCADE,
CONSTRAINT validation_result_name_uq UNIQUE (run_id, check_name)
);

-- -----
-- The data dictionary lives in the database too (loaded from docs/data_dictionary.csv).
-- v_dq_undocumented_columns compares it with the catalog, so it cannot go stale quietly.
-- -----

CREATE TABLE data_dictionary (
table_name   text NOT NULL,
column_name  text NOT NULL,
definition   text NOT NULL,
source       text NOT NULL,
allowable_values text NOT NULL,
null_meaning text NOT NULL,
sensitivity  text NOT NULL,
CONSTRAINT data_dictionary_pk PRIMARY KEY (table_name, column_name),
CONSTRAINT data_dictionary_sensitivity_ck CHECK (sensitivity ~ '^(Public|Internal|Restricted)'),

```

```

        CONSTRAINT data_dictionary_filled_ck      CHECK (definition <> '' AND source <> '' AND allowable_values <> '' AND
null_meaning <> '')
);

```

sql/02_cardinality_triggers.sql

```

-- =====
-- 02_cardinality_triggers.sql
-- The "one or more" ends of two relationships in the ER diagram:
--   a crash kills ONE OR MORE pedestrians
--   a crash involves ONE OR MORE vehicles in transport
-- A foreign key can only say "every child has a parent". It cannot say
-- "every parent has a child", so these two rules need a trigger.
--
-- The triggers are DEFERRED: they run at COMMIT, not per statement.
-- That way a load can insert the crash first and its children next,
-- as long as they arrive in the same transaction.
-- =====

CREATE FUNCTION trg_crash_min_children() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    k_year smallint;
    k_case integer;
BEGIN
    IF TG_TABLE_NAME = 'crash' THEN          -- fired by INSERT on crash
        k_year := NEW.year; k_case := NEW.st_case;
    ELSE                                     -- fired by DELETE/UPDATE on a child table
        k_year := OLD.year; k_case := OLD.st_case;
    END IF;

    -- The crash itself is gone (ON DELETE CASCADE removed the children): nothing to protect.
    IF NOT EXISTS (SELECT 1 FROM crash c WHERE c.year = k_year AND c.st_case = k_case) THEN
        RETURN NULL;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pedestrian p WHERE p.year = k_year AND p.st_case = k_case) THEN
        RAISE EXCEPTION 'crash (%, %) has no pedestrian fatality row', k_year, k_case
        USING ERRCODE = 'check_violation', CONSTRAINT = 'crash_min_one_pedestrian', TABLE = 'crash';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM vehicle v WHERE v.year = k_year AND v.st_case = k_case) THEN
        RAISE EXCEPTION 'crash (%, %) has no in-transport vehicle row', k_year, k_case
        USING ERRCODE = 'check_violation', CONSTRAINT = 'crash_min_one_vehicle', TABLE = 'crash';
    END IF;

    RETURN NULL;
END;
$$;

CREATE CONSTRAINT TRIGGER crash_min_children_on_insert
    AFTER INSERT ON crash
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW EXECUTE FUNCTION trg_crash_min_children();

CREATE CONSTRAINT TRIGGER crash_min_children_on_pedestrian_change
    AFTER DELETE OR UPDATE OF year, st_case ON pedestrian
    DEFERRABLE INITIALLY DEFERRED
    FOR EACH ROW EXECUTE FUNCTION trg_crash_min_children();

```

```
CREATE CONSTRAINT TRIGGER crash_min_children_on_vehicle_change
  AFTER DELETE OR UPDATE OF year, st_case ON vehicle
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION trg_crash_min_children();
```